A DATA-BUDGET-AWARE AND EXPLANATION-STABLE LIGHTWEIGHT DEEP LEARNING FRAMEWORK FOR PLANT DISEASE DETECTION

A PROJECT REPORT

Submitted by

A. BOWSIYA (911625405001)

in partial fulfillment for the award of the degree

of

MASTER OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

MOTHER TERASA COLLEGE OF ENGINEERING AND TECHNOLOGY, PUDUKOTTAI-622 102

ANNA UNIVERSITY: CHENNAI 600 025

NOVEMBER 2026

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report **"A Data-Budget-Aware and Explanation-Stable Lightweight Deep Learning Framework for Plant Disease Detection"** is the Bonafide work of **"A. BOWSIYA (911625405001)"** who carried out the project work under my supervision.

SIGNATURE

T. TAMILILAKKIYA M.E.,
HEAD OF THE DEPARTMENT
ASSISTANT PROFESSOR
Department of Computer Science and Engineering
Mother Terasa College of Engineering and Technology,
Pudukkottai - 622 102

SIGNATURE

Mrs. P. JONE JENIFER M.E.,
SUPERVISOR
ASSISTANT PROFESSOR
Department of Computer Science and Engineering
Mother Terasa College of Engineering and Technology,
Pudukkottai - 622 102

The project report submitted for the viva voce held on ……..

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I am much grateful to the management of Mother Terasa College of Engineering and Technology, Pudukkottai for providing me an opportunity to undergo this project work.

I have immense pleasure and satisfaction in expressing my hearty thanks to our beloved managing founder **Dr. R. CHINNATHAMBI,** who is the source of this great institution.

I am very glad to express my hearty thanks to our beloved Chairman **Mr. R.C. UTHAYAKUMAR B.A.,** who is the source of spirit and strength of this great institution.

I would like to express my sincere thanks to our respected Principal **Dr. M. RAVICHANDRAN Ph.D.,** Principal, MTCET, for the facilities and encouragement given towards the progress and completion of this project.

I am grateful to **T. TAMILILAKKIYA M.E.,** Head of the Department, Department of Computer Science and Engineering, for the effective leadership and supportive guidance given to this project.

I wish to convey my heartfelt thanks to **Mrs. P. JONE JENIFER M.E.,** Assistant Professor, Department of Computer Science and Engineering, my project guide, for not only showing the path but also encouraging me.

I wish to extend my sincere thanks to all faculty members of the Computer Science and Engineering Department for their valuable suggestions and encouragement towards the successful completion of this project.

Finally, I express my profound gratitude to the Almighty, who gave me the bravery, privilege, power and belief to complete this project.

ABSTRACT

Plant diseases reduce crop yield, and early recognition depends on scarce experts. This project develops a lightweight, explainable deep learning system that identifies plant leaf diseases from a photograph.

The PlantVillage colour dataset (54,305 images, 38 crop–disease classes covering 14 crops) was audited for unreadable files and exact duplicates and divided with a leaf-group-aware split into training (38,105), validation (8,150) and test (8,050) partitions. Controlled training budgets of 20%, 40%, 60%, 80% and 100% were sampled from the training partition only. A MobileNetV3-Small network pretrained on ImageNet was fine-tuned on the 40% budget (15,240 images) at 160 × 160 pixels, using label-preserving augmentation and class-weighted cross-entropy to counter class imbalance.

On the held-out test set of 8,050 images the model achieved an accuracy of **98.30%**, macro precision of 97.74%, macro recall of 97.78% and macro F1-score of **97.74%**, with only 1.56 million parameters, a 6.4 MB checkpoint and an average CPU inference time of 22 ms per image. Grad-CAM heatmaps explain each prediction; under horizontal flips the prediction agreed in 98.4% of cases with a mean heatmap correlation of 0.947. The model is deployed in a Flask web application with login, upload, Grad-CAM view, reports and history.

Keywords: Plant disease detection, deep learning, MobileNetV3-Small, transfer learning, limited data, Grad-CAM, explainable AI, Flask.

ஆய்வுச் சுருக்கம்

தாவர நோய்கள் பயிர் விளைச்சலைக் குறைக்கின்றன. அவற்றை ஆரம்ப நிலையிலேயே கண்டறிய வல்லுநர்களின் உதவி தேவைப்படுகிறது; ஆனால் அவர்கள் எல்லா இடங்களிலும் எளிதில் கிடைப்பதில்லை. இந்தத் திட்டம், ஓர் இலையின் புகைப்படத்திலிருந்து தாவர இலை நோய்களைக் கண்டறியும், எடை குறைந்த மற்றும் தன் முடிவுகளை விளக்கக்கூடிய ஆழ்கற்றல் (Deep Learning) அமைப்பை உருவாக்குகிறது.

PlantVillage வண்ணத் தரவுத்தொகுப்பு (54,305 படங்கள், 14 பயிர்களை உள்ளடக்கிய 38 பயிர்-நோய் வகுப்புகள்) படிக்க இயலாத கோப்புகள் மற்றும் ஒரே மாதிரியான நகல்களுக்காகச் சரிபார்க்கப்பட்டு, இலைக் குழுவைக் கருத்தில் கொண்ட பிரிப்பு முறையில் பயிற்சி (38,105), சரிபார்ப்பு (8,150), சோதனை (8,050) எனப் பிரிக்கப்பட்டது. 20%, 40%, 60%, 80%, 100% என்ற கட்டுப்படுத்தப்பட்ட பயிற்சித் தரவு அளவுகள் பயிற்சிப் பகுதியிலிருந்து மட்டுமே எடுக்கப்பட்டன. ImageNet-இல் முன்பயிற்சி பெற்ற MobileNetV3-Small வலையமைப்பு, 40% தரவு அளவில் (15,240 படங்கள்) 160 × 160 பிக்சல் அளவில் நுண்சீரமைப்பு (fine-tuning) செய்யப்பட்டது. வகுப்புச் சமநிலையின்மையைச் சமாளிக்க, லேபிளை மாற்றாத தரவுப் பெருக்கமும் (augmentation) வகுப்பு எடையிடப்பட்ட cross-entropy இழப்புச் சார்பும் பயன்படுத்தப்பட்டன.

8,050 படங்கள் கொண்ட தனியான சோதனைத் தொகுப்பில் இம்மாதிரி **98.30%** துல்லியம் (accuracy), 97.74% macro precision, 97.78% macro recall, **97.74%** macro F1-மதிப்பெண் ஆகியவற்றை அடைந்தது. இதில் 1.56 மில்லியன் அளவுருக்கள் மட்டுமே உள்ளன; மாதிரிக் கோப்பின் அளவு 6.4 MB; ஒரு படத்திற்குச் சராசரியாக 22 மில்லி விநாடிகளில் CPU-வில் கணிப்பு செய்கிறது. ஒவ்வொரு கணிப்பையும் Grad-CAM வெப்ப வரைபடங்கள் விளக்குகின்றன. படங்களைக் கிடைமட்டமாகப் புரட்டியபோது 98.4% நேரங்களில் கணிப்பு மாறவில்லை; வெப்ப வரைபடங்களின் சராசரித் தொடர்பு 0.947 ஆக இருந்தது. இம்மாதிரி உள்நுழைவு, படப் பதிவேற்றம், Grad-CAM காட்சி, அறிக்கைகள், வரலாறு ஆகிய வசதிகளுடன் கூடிய Flask இணையப் பயன்பாட்டில் செயல்படுத்தப்பட்டுள்ளது.

முக்கியச் சொற்கள்: தாவர நோய் கண்டறிதல், ஆழ்கற்றல், MobileNetV3-Small, இடமாற்றக் கற்றல் (Transfer Learning), குறைந்த தரவு, Grad-CAM, விளக்கக்கூடிய செயற்கை நுண்ணறிவு, Flask.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	i
	ஆய்வுச் சுருக்கம்	ii
	LIST OF FIGURES	v
	LIST OF TABLES	vi
	LIST OF ABBREVIATIONS	vii
1	**INTRODUCTION**	1
	1.1 DEEP LEARNING	1
	1.2 IMAGE PROCESSING	2
	1.3 EXPLAINABLE ARTIFICIAL INTELLIGENCE	3
	1.4 PROBLEM STATEMENT	3
	1.5 OBJECTIVES	3
	1.6 ORGANISATION OF THE REPORT	4
2	**LITERATURE SURVEY**	5
3	**SYSTEM STUDY**	9
	3.1 EXISTING SYSTEM	9
	3.2 PROPOSED SYSTEM	10
4	**SYSTEM REQUIREMENTS**	11
	4.1 HARDWARE REQUIREMENTS	11
	4.2 SOFTWARE REQUIREMENTS	11
5	**SYSTEM DESIGN**	12
	5.1 SYSTEM ARCHITECTURE	12
	5.2 UML DIAGRAMS	12
	5.3 DATA FLOW DIAGRAM	16
	5.4 DATABASE DESIGN	18
6	**SOFTWARE DESCRIPTION**	20
	6.1 FRONT END	20
	6.2 BACK END	21
	6.3 DATABASE – SQLITE	22

CHAPTER NO	TITLE	PAGE NO
7	**SYSTEM IMPLEMENTATION**	23
	7.1 MODULE LIST	23
	7.2 MODULE DESCRIPTION	23
8	**SYSTEM TESTING**	28
	8.1 SYSTEM TESTING	28
	8.2 TEST CASES	29
	8.3 RESULTS AND DISCUSSION	30
9	**CONCLUSION AND FUTURE ENHANCEMENT**	35
	9.1 CONCLUSION	35
	9.2 FUTURE ENHANCEMENT	35
	APPENDIX 1	37
	A1.1 APP.PY (Flask web application)	37
	A1.2 PREDICT.PY (model bundle loader)	44
	A1.3 GRADCAM.PY (explanation module)	46
	A1.4 TRAIN.PY (training pipeline)	47
	A1.5 RESULT.HTML (result page template)	58
	APPENDIX 2	60
	REFERENCES	65

LIST OF FIGURES

FIG NO	TITLE	PAGE NO
5.1	System Architecture	12
5.2.1	Use Case Diagram	13
5.2.2	Class Diagram	14
5.2.3	Sequence Diagram	15
5.2.4	Activity Diagram	16
5.3.1	Level 0 Data Flow Diagram	17
5.3.2	Level 1 Data Flow Diagram	17
5.3.3	Level 2 Data Flow Diagram (Prediction process)	18
7.2.1	Class distribution of the PlantVillage dataset	24
8.1.5	Flow of testing	29
8.3.1	Training and validation curves	31
8.3.2	Per-class F1-score on the test set	32
8.3.3	Confusion matrix (row-normalised)	33
8.3.4	Grad-CAM explanations for test images	33
A2.1	Login page	60
A2.2	Registration page	61
A2.3	Login with invalid credentials	61
A2.4	Leaf image upload page	62
A2.5	Prediction result – Tomato Late blight with Grad-CAM	62
A2.6	Prediction result – healthy Apple leaf	63
A2.7	Printable prediction report	63
A2.8	Prediction history	64

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
5.1	users table	18
5.2	predictions table	19
7.1	Controlled training budgets	24
7.2	Training configuration	25
8.1	Test case summary	30
8.2	Test-set performance of the proposed model	31
8.3	Explanation stability (380 test images)	34
8.4	Comparison with published PlantVillage results	34

LIST OF ABBREVIATIONS

AI	–	Artificial Intelligence
CNN	–	Convolutional Neural Network
CPU	–	Central Processing Unit
CSS	–	Cascading Style Sheets
DFD	–	Data Flow Diagram
F1	–	Harmonic mean of precision and recall
GPU	–	Graphics Processing Unit
Grad-CAM	–	Gradient-weighted Class Activation Mapping
HTML	–	HyperText Markup Language
IoU	–	Intersection over Union
ML	–	Machine Learning
ReLU	–	Rectified Linear Unit
SQL	–	Structured Query Language
SVM	–	Support Vector Machine
UML	–	Unified Modeling Language
XAI	–	Explainable Artificial Intelligence

CHAPTER 1
INTRODUCTION

Agriculture supports the livelihood of a large part of the population, and plant diseases are one of the main causes of loss in crop yield and quality. Fungal, bacterial and viral infections usually appear first on the leaves as spots, lesions, mildew, blight, mosaic patterns, curling or yellowing. When these symptoms are recognised early, the farmer can isolate infected plants and seek appropriate advice before the disease spreads across the field.

The conventional way of identifying a disease is visual inspection by the farmer or by an agricultural expert. Expert inspection is reliable but is not always available at the right time, especially in remote villages and during peak seasons when many fields need attention. Different diseases can also produce visually similar symptoms, so the decision depends heavily on experience. Laboratory confirmation is accurate but slow and costly for routine screening.

Advances in computer vision now make it possible to recognise disease symptoms automatically from a photograph of a leaf. A deep convolutional neural network can learn the colour, texture and shape patterns of each disease from labelled images and then classify a new image within a fraction of a second. This project builds such a system with three practical goals: it must be **lightweight** enough to run on an ordinary computer, it must remain reliable when the amount of labelled training data is limited, and it must **explain** its decisions so that users can check whether the model looked at the diseased region of the leaf.

1.1 DEEP LEARNING

Deep learning is a branch of machine learning in which models built from many layers of artificial neurons learn useful representations directly from raw data. In a convolutional neural network (CNN), the early layers detect simple

patterns such as edges and colour blobs, the middle layers combine them into textures and lesion shapes, and the final layers map these features to class scores. Because the features are learned rather than designed by hand, CNNs adapt well to the large variety of disease symptoms found on different crops.

Training a deep network from scratch requires a large dataset. **Transfer learning** reduces this requirement by starting from a network that has already learned general visual features on the ImageNet dataset and fine-tuning it on the plant-disease images. **Lightweight architectures** such as MobileNetV3-Small use depthwise separable convolutions, inverted residual blocks and squeeze-and-excitation attention to keep the number of parameters and the computation low, which makes them suitable for laptops, edge devices and web servers without a GPU.

1.2 IMAGE PROCESSING

Image processing prepares the raw photograph so that the network receives consistent input. In this project each uploaded image is decoded and converted to the RGB colour space, resized so that its shorter side is 184 pixels, centre-cropped to 160 × 160 pixels, converted to a tensor and normalised with the ImageNet channel mean and standard deviation. The same deterministic pipeline is used during validation, testing and in the web application, so the model always sees images prepared in the same way.

During training, label-preserving **data augmentation** creates varied views of each training image: random resized cropping, horizontal flipping and mild changes of brightness, contrast, saturation and hue. These transformations imitate natural variation in camera position and lighting without removing the disease symptoms, which helps the model generalise when the training budget is small.

1.3 EXPLAINABLE ARTIFICIAL INTELLIGENCE

A prediction is more useful when the user can see why it was made. Gradient-weighted Class Activation Mapping (Grad-CAM) uses the gradients of the predicted class score with respect to the last convolutional feature maps to produce a coarse heatmap of the image regions that most influenced the decision. The heatmap is overlaid on the leaf so that users can confirm that the model attended to the lesions rather than to the background. This project also measures how stable these explanations remain when the same leaf is flipped, brightened or slightly rotated.

1.4 PROBLEM STATEMENT

Given leaf images belonging to 38 crop–disease classes, the problem is to build a compact image classifier that (i) achieves high accuracy and balanced performance across frequent and rare classes when trained on a limited, controlled share of the labelled data, (ii) provides stable visual explanations of its predictions, and (iii) is available to end users through a simple, secure web interface.

1.5 OBJECTIVES

- To audit the PlantVillage dataset and create a leakage-free, leaf-group-aware training, validation and test split with controlled training budgets.
- To design a preprocessing and augmentation pipeline that preserves disease symptoms.
- To fine-tune a lightweight MobileNetV3-Small classifier with class-weighted loss and evaluate it with accuracy, macro precision, macro recall and macro F1-score.
- To generate Grad-CAM explanations and quantify their stability under label-preserving transformations.

- To deploy the model in a Flask web application with authentication, prediction, explanation, report and history modules.

1.6 ORGANISATION OF THE REPORT

Chapter 2 reviews related work. Chapter 3 studies the existing and proposed systems. Chapter 4 lists the hardware and software requirements. Chapter 5 presents the system architecture, UML diagrams and data flow diagrams. Chapter 6 describes the software used. Chapter 7 explains the implementation modules. Chapter 8 covers testing and results, and Chapter 9 concludes the report with future enhancements. Appendix 1 contains the source code and Appendix 2 the screenshots of the application.

CHAPTER 2
LITERATURE SURVEY

[1] Hughes, D. P. and Salathé, M. (2015) – "An open access repository of images on plant health to enable the development of mobile disease diagnostics"

Description: The authors released the PlantVillage dataset, containing more than 50,000 expertly curated images of healthy and infected leaves of 14 crop species across 38 classes. The images were captured on a uniform background under controlled conditions, and the dataset was made openly available to encourage research on automatic, smartphone-based disease diagnosis. PlantVillage has since become the standard benchmark for plant disease classification and is the dataset used in this project.

Drawbacks: Images were taken in controlled settings with plain backgrounds, so models trained only on this dataset may not transfer directly to cluttered field photographs.

[2] Mohanty, S. P., Hughes, D. P. and Salathé, M. (2016) – "Using deep learning for image-based plant disease detection"

Description: This study trained AlexNet and GoogLeNet on 54,306 PlantVillage images and compared training from scratch with transfer learning, as well as colour, grey-scale and segmented versions of the images. The best configuration, GoogLeNet with transfer learning, reached 99.35% accuracy on a held-out test set, showing that deep CNNs can recognise crop–disease pairs very accurately.

Drawbacks: When the trained model was tested on images collected from other sources, accuracy dropped sharply (to about 31%), revealing a strong dependence on the capture conditions of the training data. The networks used are also large for mobile deployment.

[3] Sladojevic, S., Arsenovic, M., Anderla, A., Culibrk, D. and Stefanovic, D. (2016) – "Deep neural networks based recognition of plant diseases by leaf image classification"

Description: The authors collected leaf images from the internet, augmented them, and fine-tuned a CaffeNet model to recognise 13 types of plant disease plus healthy leaves and background. The model achieved an average precision of 96.3%, demonstrating that CNNs trained on augmented web images can distinguish multiple diseases.

Drawbacks: The dataset was relatively small and assembled from web searches, the number of classes was limited, and the model gave no explanation of its decisions.

[4] Brahimi, M., Boukhalfa, K. and Moussaoui, A. (2017) – "Deep learning for tomato diseases: classification and symptoms visualization"

Description: Using about 14,800 tomato leaf images from PlantVillage covering nine diseases, the authors fine-tuned AlexNet and GoogLeNet and obtained up to 99.18% accuracy. They also visualised the regions used by the network, showing that the learned features correspond to visible symptoms, which increases trust in the model.

Drawbacks: The work is restricted to a single crop, and the visualisation was qualitative; the stability of the visualisations was not measured.

[5] Ferentinos, K. P. (2018) – "Deep learning models for plant disease detection and diagnosis"

Description: Several CNN architectures were trained on an open database of 87,848 images covering 25 plant species in 58 classes, including images captured in real cultivation conditions. The VGG model achieved the best success rate of 99.53%, and the study discussed the potential of such models as early-warning tools.

Drawbacks: VGG has well over 100 million parameters, making it heavy for low-cost devices, and performance was lower when training on laboratory images and testing on field images.

[6] Barbedo, J. G. A. (2018) – "Impact of dataset size and variety on the effectiveness of deep learning and transfer learning for plant disease classification"

Description: This study analysed how the size and variety of the training data influence CNN accuracy for plant disease recognition. It showed that small and insufficiently diverse datasets lead to poor generalisation, and that transfer learning reduces but does not remove the dependence on data quantity.

Drawbacks: The study highlights the problem but does not propose a controlled protocol for measuring performance at different training budgets, which this project adopts.

[7] Too, E. C., Yujian, L., Njuki, S. and Yingchun, L. (2019) – "A comparative study of fine-tuning deep learning models for plant disease identification"

Description: VGG-16, Inception-v4, ResNet-50, ResNet-101, ResNet-152 and DenseNet-121 were fine-tuned on PlantVillage. DenseNet-121 achieved the highest test accuracy of 99.75% while using fewer parameters than the other deep models and without overfitting.

Drawbacks: All compared networks are still considerably larger than mobile architectures, and the evaluation used the full training set only.

[8] Howard, A. et al. (2019) – "Searching for MobileNetV3"

Description: MobileNetV3 was designed using hardware-aware neural architecture search combined with the NetAdapt algorithm, inverted residual blocks, squeeze-and-excitation modules and the hard-swish activation. The

Small variant targets low-resource devices and improves accuracy and latency compared with MobileNetV2 on ImageNet.

Drawbacks: The architecture was evaluated on general image classification; its behaviour on plant-disease data and under limited training data must be measured separately.

[9] Selvaraju, R. R. et al. (2017) – "Grad-CAM: visual explanations from deep networks via gradient-based localization"

Description: Grad-CAM produces class-discriminative localisation maps for any CNN by weighting the final convolutional feature maps with the spatially averaged gradients of the target class score. It requires no change to the network architecture and is widely used to inspect what image regions drive a prediction.

Drawbacks: The maps are coarse (limited by the final feature-map resolution) and do not prove causality; their consistency under small image changes is not guaranteed.

[10] Pal, C., Karmakar, S., Mukherjee, I. and Chakrabarti, P. P. (2025) – "A lightweight and explainable CNN model for empowering plant disease diagnosis"

Description: The authors proposed Mob-Res, a parallel network combining a MobileNetV2 feature path with a residual convolution path, using about 3.51 million parameters. Evaluated on the Plant Disease Expert and PlantVillage datasets, it reported average accuracies of 97.73% and 99.47% respectively, and used Grad-CAM, Grad-CAM++ and LIME to explain predictions.

Drawbacks: The explanations are presented qualitatively, and model behaviour under reduced training data is not studied. This project addresses both points with a smaller backbone (1.56 million parameters).

CHAPTER 3
SYSTEM STUDY

3.1 EXISTING SYSTEM

In the existing approach, plant diseases are identified mainly by manual visual inspection carried out by farmers or agricultural extension officers. The farmer observes symptoms on the leaves and either relies on personal experience or carries samples to an expert or a laboratory. Laboratory methods such as microscopy, culturing and molecular tests are accurate but need trained staff, equipment and several days.

Automatic systems reported in the literature mostly train large CNNs such as AlexNet, VGG or ResNet on the full PlantVillage training set and report a single accuracy value. Such systems do not show how performance changes when fewer labelled images are available, often report only overall accuracy (which hides weak minority classes), are heavy for low-cost hardware, and present a prediction without any explanation that a user could verify.

DISADVANTAGES

- Manual inspection is time consuming and depends on the availability and experience of experts.
- Similar-looking symptoms of different diseases lead to misidentification.
- Large deep models need high memory and computation, limiting deployment.
- Performance under limited labelled data and on rare classes is not measured.
- Predictions are given as a black box without visual explanation.
- Data leakage between training and test images can inflate reported accuracy.

3.2 PROPOSED SYSTEM

The proposed system is a lightweight, explainable, web-based plant disease detection system. The dataset is audited and split by leaf groups before any augmentation, so that near-identical photographs of the same leaf never appear in both training and test data. Training is performed on a controlled 40% budget of the training partition to reflect a limited-data setting. A pretrained MobileNetV3-Small network is fine-tuned with label-preserving augmentation and class-weighted cross-entropy loss, and the best checkpoint is selected by validation macro F1-score.

The trained model is exported as a versioned bundle and served by a Flask web application. A registered user uploads a leaf photograph; the system validates the file, predicts the top-5 classes with confidence, highlights the decisive regions with Grad-CAM, warns when confidence is low, and stores the result in the user's history from which a printable report can be produced.

ADVANTAGES

- Fast prediction (about 22 ms per image on a CPU) from a single leaf photograph.
- High accuracy (98.30%) and balanced macro F1-score (97.74%) on unseen test images.
- Lightweight model with 1.56 million parameters – no GPU needed for inference.
- Grad-CAM heatmap shows why a prediction was made; stability is quantified.
- Secure user accounts, prediction history and printable reports.
- Leakage-free evaluation protocol with fixed seeds for reproducibility.

CHAPTER 4
SYSTEM REQUIREMENTS

4.1 HARDWARE REQUIREMENTS

System	:	PC or Laptop
Processor	:	Intel Core i5 or higher
RAM	:	8 GB (16 GB recommended for training)
Hard Disk	:	20 GB free space
GPU	:	Optional (the model was trained and runs on CPU)
Monitor	:	1366 × 768 or higher resolution

4.2 SOFTWARE REQUIREMENTS

Operating System	:	Windows 10 / 11
Front End	:	HTML, CSS, Jinja2 templates
Back End	:	Python 3.12, Flask 3.1
Deep Learning	:	PyTorch 2.3, torchvision 0.18
Libraries	:	Pillow, NumPy, Matplotlib
Database	:	SQLite 3
IDE	:	Visual Studio Code
Browser	:	Google Chrome / Microsoft Edge

CHAPTER 5
SYSTEM DESIGN

5.1 SYSTEM ARCHITECTURE

System architecture describes the high-level structure of the software system, its components and the way they interact. The proposed system has two parts. The offline training pipeline audits and splits the dataset, applies preprocessing and augmentation, trains the MobileNetV3-Small classifier and exports a model bundle containing the weights, the class list and the preprocessing configuration. The online Flask web application loads this bundle once at start-up and serves authentication, image upload, prediction with Grad-CAM, reports and history, storing users and predictions in an SQLite database.

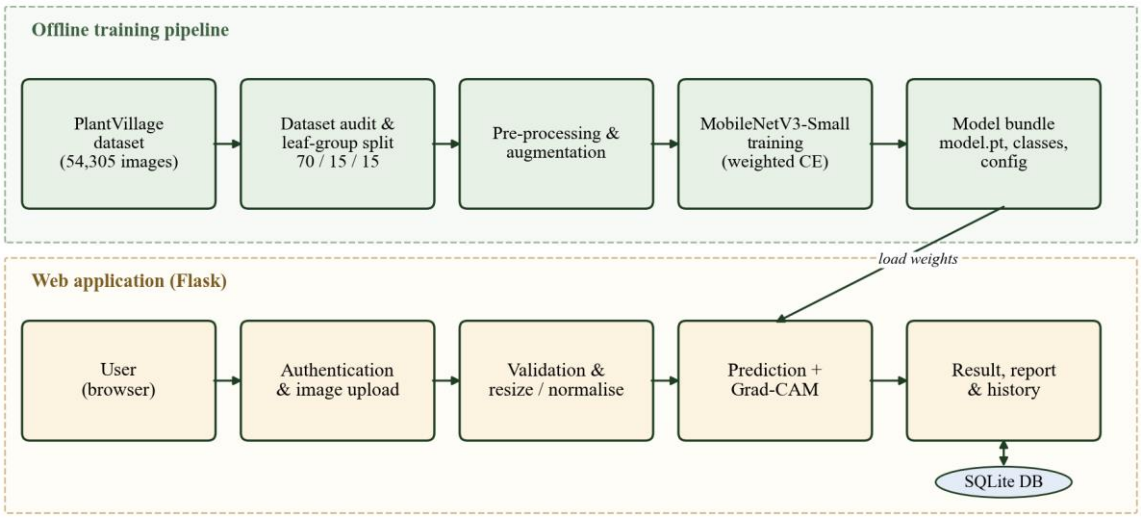


Fig 5.1 System Architecture

5.2 UML DIAGRAMS

A UML (Unified Modeling Language) diagram visually represents a system together with its main actors, roles, actions, artefacts and classes, in order to better understand, alter, maintain or document the system. The following UML diagrams describe the proposed system:

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Activity Diagram

5.2.1 Use Case Diagram

A use case diagram shows the interactions between actors and the system. The primary actor is the user (a farmer, student or agricultural officer) who registers, logs in, uploads a leaf image, views the prediction, confidence and Grad-CAM heatmap, prints the report, reviews the history and logs out. The trained model acts as a secondary actor that supplies the prediction and explanation.

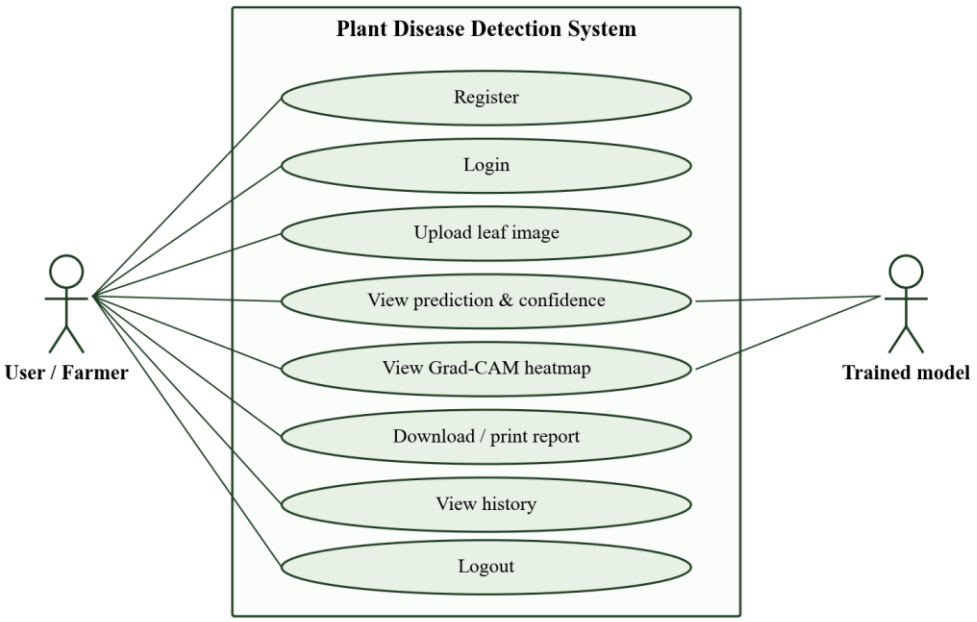


Fig 5.2.1 Use Case Diagram

5.2.2 Class Diagram

A class diagram is a static structure diagram that describes the classes of a system, their attributes, operations and relationships.

- **LeafDataset:** loads images listed in the split manifest and applies the transforms.
- **MobileNetV3Small:** the CNN with a 38-way classification head.
- **Trainer:** builds the model, runs the training loop with weighted loss and selects the best checkpoint.
- **GradCAM:** hooks the last convolutional block to compute and overlay heatmaps.
- **FlaskApp:** handles the web routes for authentication, prediction, reports and history.
- **Database:** the users and predictions tables in SQLite.

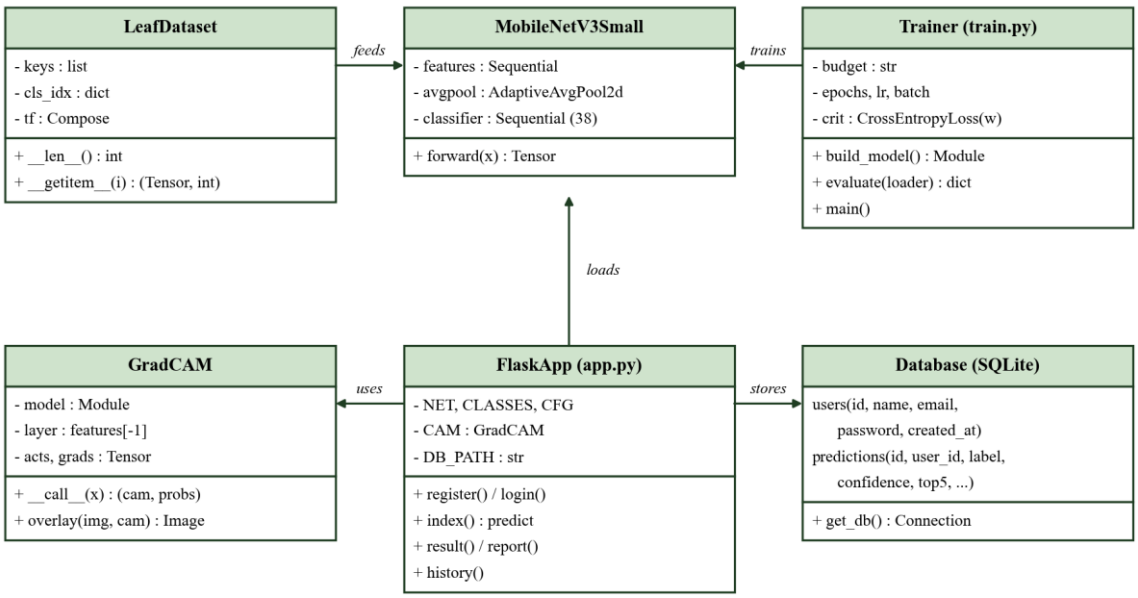


Fig 5.2.2 Class Diagram

5.2.3 Sequence Diagram

A sequence diagram shows object interactions arranged in time order. The user logs in through the browser, the Flask server verifies the hashed password from the database, the uploaded image is validated and resized, the model and Grad-CAM produce the probabilities and heatmap, the prediction is inserted into the database and the result page is returned to the user.

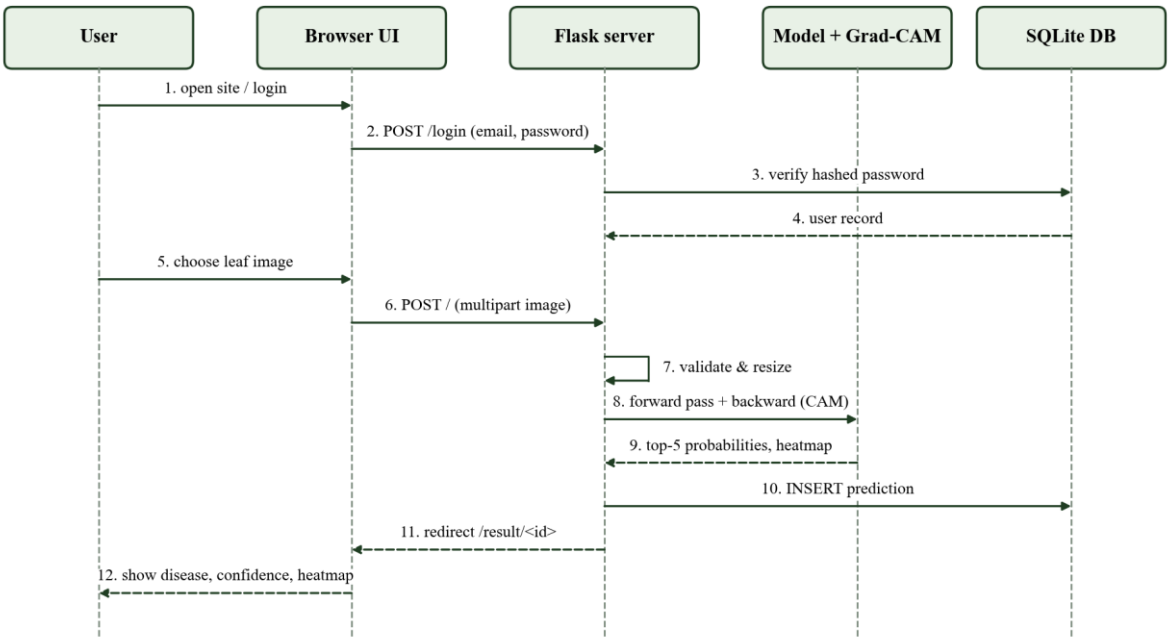


Fig 5.2.3 Sequence Diagram

5.2.4 Activity Diagram

An activity diagram represents the flow of control from one activity to another. Invalid uploads (unsupported type, unreadable or very small images) return the user to the upload step with an error message; valid images continue through preprocessing, prediction, explanation and storage.

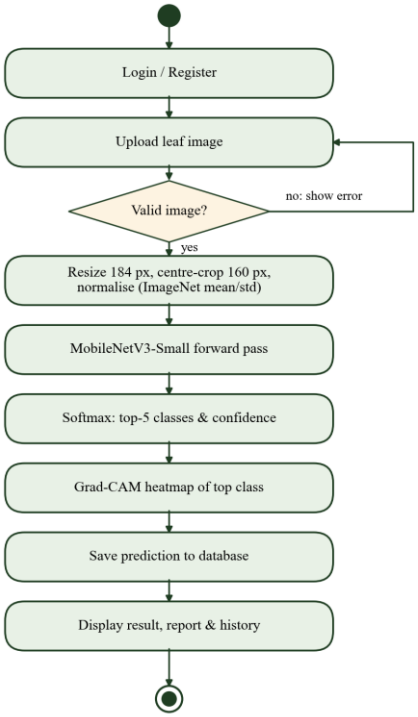


Fig 5.2.4 Activity Diagram

5.3 DATA FLOW DIAGRAM

A data flow diagram shows how data is processed and transferred in a system. It identifies the external entities, processes and data stores, and how the inputs are transformed into outputs. Level 0 shows the whole system as a single process; Level 1 divides it into its main processes; Level 2 expands the prediction process.

5.3.1 Level 0 Data Flow Diagram

The Level 0 diagram (context diagram) shows the whole system as a single process. The user supplies the leaf image and login credentials and receives the disease, confidence, Grad-CAM heatmap and report.

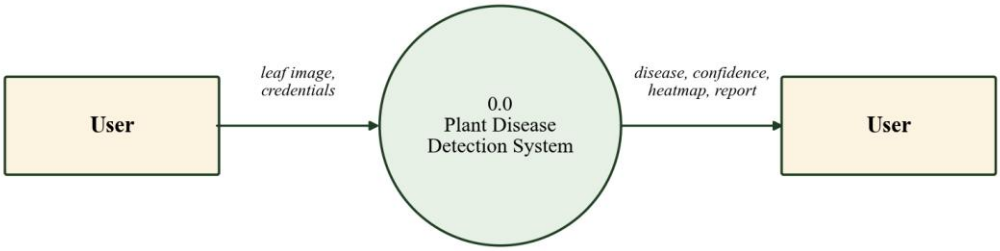


Fig 5.3.1 Level 0 Data Flow Diagram

5.3.2 Level 1 Data Flow Diagram

The Level 1 diagram divides the system into four processes: authenticate, upload and validate, predict and explain, and report and history. They use three data stores: users (D1), the model bundle (D2) and predictions (D3).

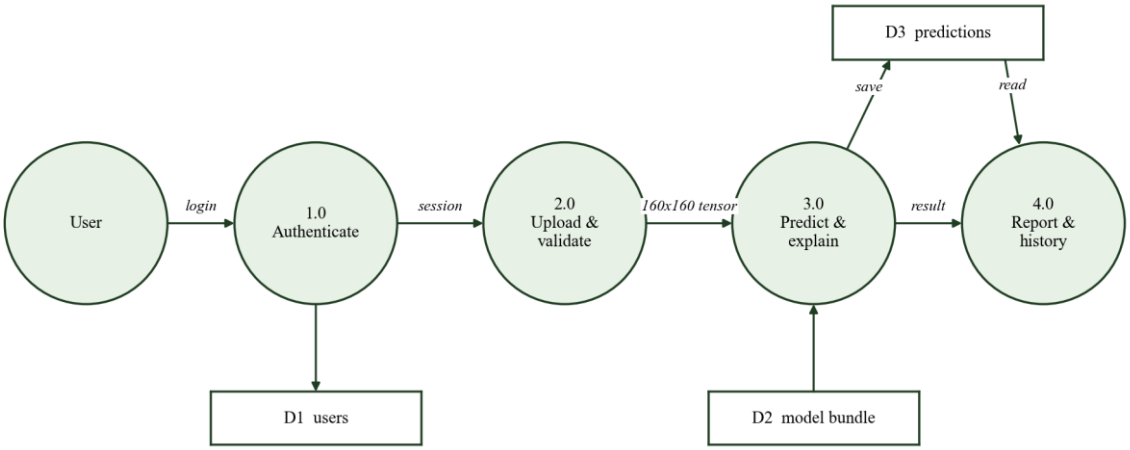


Fig 5.3.2 Level 1 Data Flow Diagram

5.3.3 Level 2 Data Flow Diagram

The Level 2 diagram expands process 3.0 (predict and explain): the image is resized, cropped and normalised, passed forward through the CNN using the stored model weights, converted to top-5 labels by softmax and explained by Grad-CAM, and the labels and heatmap are then stored and displayed.

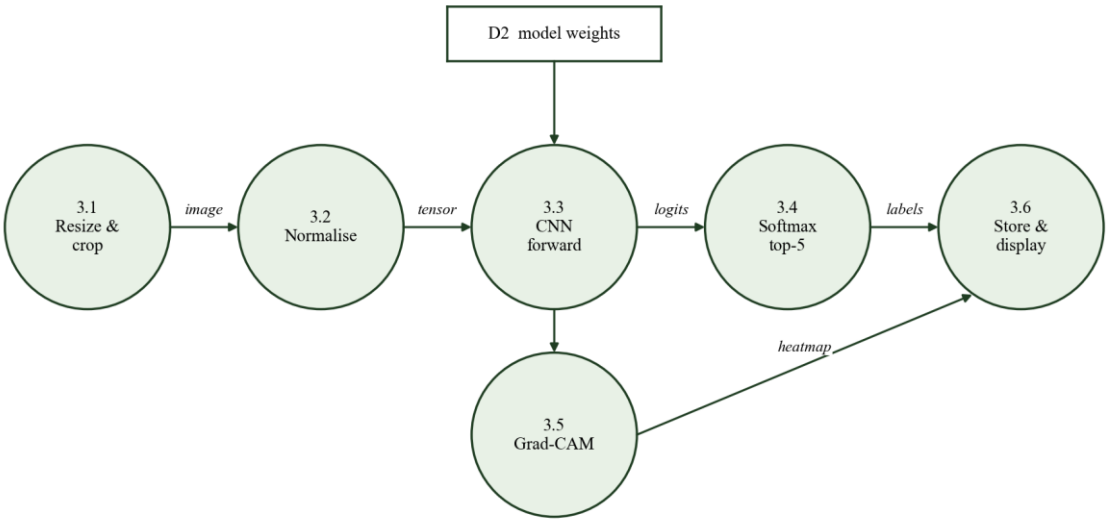


Fig 5.3.3 Level 2 Data Flow Diagram (Prediction process)

5.4 DATABASE DESIGN

The application uses an SQLite database with two tables. Passwords are never stored in plain text; they are hashed with the scrypt algorithm provided by Werkzeug.

Field	Type	Description
id	INTEGER (PK)	Auto-increment user identifier
name	TEXT	Full name
email	TEXT (UNIQUE)	Login identifier
password	TEXT	Salted password hash
created_at	TEXT	Registration timestamp

Table 5.1 users table

Field	Type	Description
id	INTEGER (PK)	Prediction / report number
user_id	INTEGER (FK)	Owner of the prediction
image_file, cam_file	TEXT	Stored input and Grad-CAM image names
orig_name	TEXT	Original upload file name
label	TEXT	Top-1 class
confidence	REAL	Top-1 probability
top5	TEXT	Top-5 class indices and probabilities
latency_ms	REAL	Inference + Grad-CAM time
created_at	TEXT	Prediction timestamp

Table 5.2 predictions table

CHAPTER 6
SOFTWARE DESCRIPTION

6.1 FRONT END

6.1.1 HTML

HTML (HyperText Markup Language) is the standard markup language for creating web pages. It describes the structure of a page using elements such as headings, paragraphs, forms, images and tables. In this project HTML templates define the login, registration, upload, result, report and history pages. Form elements collect user credentials and the leaf image, and the file input restricts uploads to image types.

6.1.2 CSS

CSS (Cascading Style Sheets) controls the presentation of HTML documents, including colours, fonts, spacing and layout. A single stylesheet gives the application a consistent green agricultural theme, a responsive grid layout that adapts to phone screens, confidence bars for the top-5 predictions and a print style that hides navigation elements so that the report can be printed or saved as PDF.

6.1.3 Jinja2 Templates

Jinja2 is the template engine used by Flask. It allows Python values such as the predicted class, confidence and history records to be inserted into HTML pages, supports template inheritance through a common base layout, and escapes user-supplied text automatically, which protects the pages from script injection.

6.2 BACK END

6.2.1 Python

Python is a high-level, interpreted, dynamically typed programming language created by Guido van Rossum and first released in 1991. Its clear syntax, large standard library and rich ecosystem of scientific packages have made it the most widely used language for machine learning. In this project Python is used for every stage: dataset auditing, splitting, training, evaluation, explanation analysis, figure generation and the web server, which keeps the whole pipeline consistent and reproducible.

6.2.2 PyTorch and torchvision

PyTorch is an open-source deep learning framework that provides tensors with automatic differentiation and a flexible, Python-first way of defining neural networks. torchvision supplies image transforms and pretrained models, including MobileNetV3-Small with ImageNet weights. PyTorch's forward and backward hooks give direct access to feature maps and gradients, which are required for Grad-CAM.

6.2.3 Flask

Flask is a lightweight Python web framework built on the Werkzeug toolkit and the Jinja2 template engine. It maps URLs to Python functions (routes), handles form and file uploads, manages signed session cookies for logged-in users and provides secure password hashing helpers. The trained model is loaded once when the Flask application starts, so each request only performs inference.

6.2.4 Supporting Libraries

Pillow decodes and validates uploaded images and performs resizing and colour adjustments. **NumPy** supports numerical operations. **Matplotlib** is used

to draw training curves, confusion matrices, the design diagrams in this report and the colour map for Grad-CAM overlays.

6.3 DATABASE – SQLITE

SQLite is a self-contained, serverless, zero-configuration SQL database engine that stores the whole database in a single file. It is included in the Python standard library through the sqlite3 module. SQLite is well suited for this application because it requires no separate database server, supports standard SQL with transactions and constraints such as UNIQUE and FOREIGN KEY, and can be replaced with MySQL or PostgreSQL later without changing the application logic.

CHAPTER 7
SYSTEM IMPLEMENTATION

7.1 MODULE LIST

- Dataset Acquisition and Limited-Data Management module
- Preprocessing and Augmentation module
- Model Training module
- Explainable AI (Grad-CAM) module
- User Authentication module
- Prediction, Report and History module

7.2 MODULE DESCRIPTION

7.2.1 Dataset Acquisition and Limited-Data Management Module

The PlantVillage colour dataset was downloaded from its public repository. It contains 54,305 images of 14 crops in 38 classes (26 diseases and 12 healthy classes). Every file was decoded to check readability: 54,305 files were accepted and 0 rejected. Exact-duplicate detection found 21 duplicate groups containing 42 files.

PlantVillage contains several photographs of the same physical leaf. To avoid data leakage, images were grouped by leaf (40,347 groups) and whole groups were assigned to one partition, giving a 70 / 15 / 15 split with seed 42. Duplicate groups were checked to ensure that no group crosses partitions. Training budgets were then sampled per class from the training partition only, while validation and test sets stay fixed.

Budget	Fraction	Training images	Validation	Test
B20	20%	7,622	8,150	8,050
B40	40%	15,240	8,150	8,050
B60	60%	22,865	8,150	8,050
B80	80%	30,483	8,150	8,050
B100	100%	38,105	8,150	8,050

Table 7.1 Controlled training budgets

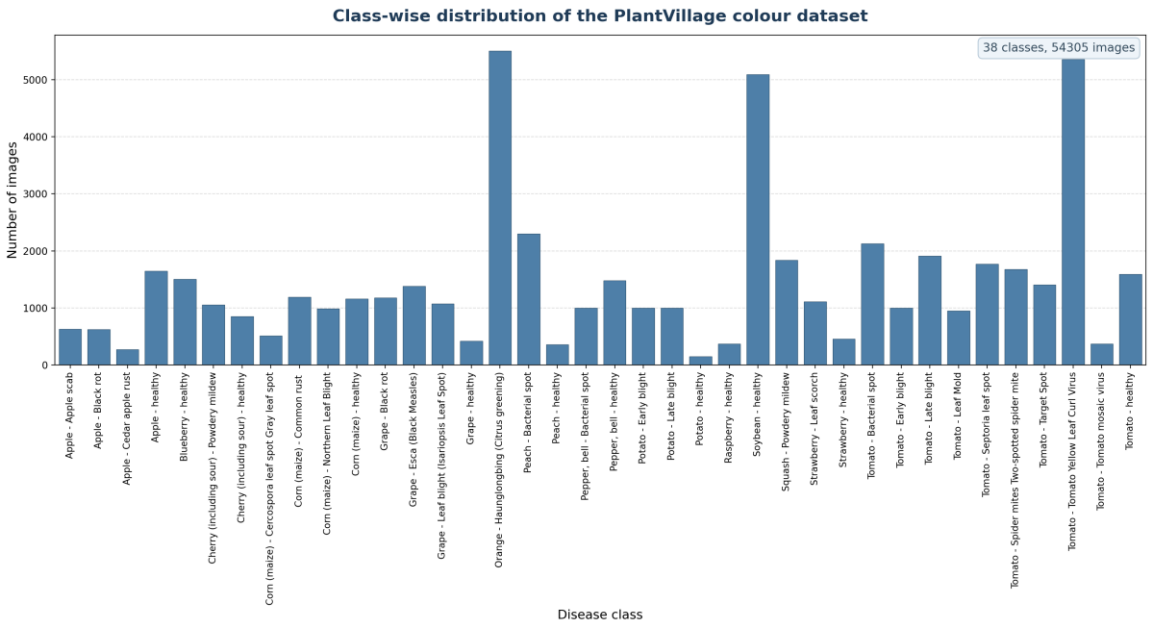


Fig 7.2.1 Class distribution of the PlantVillage dataset

7.2.2 Preprocessing and Augmentation Module

All images are converted to RGB. For evaluation and inference, each image is resized to 184 pixels on the shorter side, centre-cropped to 160 × 160 and normalised with the ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225). For training, the following label-preserving augmentations are applied on the fly:

- Random resized crop to 160 × 160 covering 60–100% of the image area.
- Random horizontal flip with probability 0.5.
- Colour jitter: brightness, contrast and saturation ±25%, hue ±0.10.
- Augmentation is applied to the training partition only.

Class imbalance (the smallest class, Potato – healthy, has 152 images while the largest, Orange – Haunglongbing (Citrus greening), has 5,507) is handled with class-weighted cross-entropy. Each class weight is proportional to the inverse square root of its training count and normalised to a mean of one, so rare classes contribute more to the loss without over-correcting.

7.2.3 Model Training Module

MobileNetV3-Small pretrained on ImageNet is used as the feature extractor. Its final 1000-way linear layer is replaced by a 38-way layer, giving 1,556,806 trainable parameters in total. The model was trained on the B40 budget (15,240 images) with the following settings:

Parameter	Value
Input size	160 × 160 × 3
Optimiser	AdamW (weight decay 1e-4)
Initial learning rate	3×10^{-4} with cosine annealing
Batch size	32
Epochs	5
Loss	Class-weighted cross-entropy
Gradient clipping	Norm 1.0
Checkpoint selection	Best validation macro F1-score
Random seed	42
Device	CPU (30.2 minutes total)

Table 7.2 Training configuration

7.2.4 Explainable AI (Grad-CAM) Module

For an input image, the module records the activations A^k of the last convolutional block of MobileNetV3-Small (576 feature maps of size 5 × 5 for a 160-pixel input) and the gradients of the predicted class score y^c with respect to them. The importance of each feature map is the spatial average of its gradient, $\alpha_k^c = (1/Z) \Sigma_i \Sigma_j \partial y^c / \partial A^k_{ij}$, and the class activation map is $L^c = \mathrm{ReLU}(\Sigma_k \alpha_k^c A^k)$. The map is normalised to [0, 1], upsampled to

the image size, coloured with the jet colour map and blended with the input image.

To measure explanation stability, 380 test images (10 per class) were explained in their original form and after a label-preserving transformation. The transformed map was mapped back to the original geometry and compared using the Pearson correlation and the IoU of the 20% most activated pixels; prediction agreement was also recorded.

7.2.5 User Authentication Module

New users register with their name, email address and a password of at least six characters. The password is stored as a salted hash; duplicate email addresses are rejected by a UNIQUE constraint. During login the hash is verified and the user identifier is stored in a signed session cookie. All prediction, report and history pages require login, and every database query is filtered by the logged-in user so that a user can never open another user's results.

7.2.6 Prediction, Report and History Module

The uploaded file is checked for an allowed extension (JPG, PNG, BMP, WEBP), a maximum size of 8 MB, a valid image signature and a minimum size of 32 × 32 pixels. The image is preprocessed and passed to the model; Grad-CAM is computed in the same pass. The top-5 classes and probabilities, the heatmap and the processing time are stored with a server-generated random file name. The result page shows the crop, condition and confidence, and flags predictions below 50% confidence as uncertain because the model can only choose among the 38 known classes.

- **Result page:** prediction, confidence, input image, Grad-CAM heatmap, top-5 bars.

- **Report page:** printable summary that can be saved as PDF from the browser.
- **History page:** all previous predictions of the user with date and time.

CHAPTER 8
SYSTEM TESTING

8.1 SYSTEM TESTING

Software testing is the process of executing a program with the intent of finding errors and verifying that it meets its requirements. Testing begins at the level of individual functions and proceeds to the integrated system. Both the machine-learning pipeline and the web application were tested.

Testing Objectives

1. Testing is a process of executing a program with the intent of finding an error.
2. A good test case has a high probability of finding an undiscovered error.
3. A successful test is one that uncovers an undiscovered error.

8.1.1 Unit Testing

Unit testing verifies individual units of code in isolation. Image loading, rejection of corrupt files, class-label lookup, split overlap checks, budget generation, augmentation output, model inference shape and Grad-CAM output were each tested separately (test cases TC-01 to TC-08).

8.1.2 Functional Testing

Functional testing checks that each feature behaves as specified. The web application's registration, login, upload validation, prediction, report and history functions were exercised with valid and invalid inputs (test cases TC-09 to TC-16).

8.1.3 Integration Testing

Integration testing combines modules and tests them as a group. The complete flow – registration, login, upload, model inference with Grad-CAM,

database insert, result page, report and history – was executed end-to-end through a real browser using an automated Playwright script, which also produced the screenshots in Appendix 2.

8.1.4 Acceptance Testing

Acceptance testing determines whether the system satisfies the user requirements. The application was checked against the objectives in Section 1.5: correct predictions on unseen test images, a visible explanation for every prediction, clear error messages and private per-user history.

8.1.5 Flow of Testing

The testing activities were carried out in the order shown below, from unit testing of individual functions to acceptance testing of the complete application.

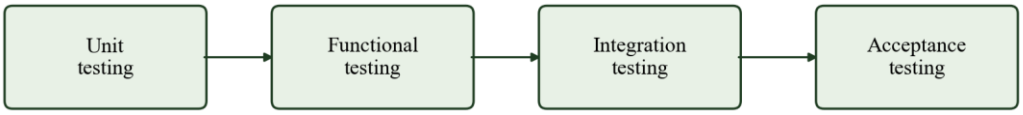


Fig 8.1.5 Flow of testing

8.2 TEST CASES

ID	Test case	Observed result	Status
TC-01	Load a valid RGB image	decoded 1 image as RGB (256, 256)	Pass
TC-02	Load a corrupt image file	1 corrupt rejected, 1 valid loaded, run continued	Pass
TC-03	Encounter an unknown class label	38 class folders known; unknown label reported	Pass
TC-04	Check partition overlap	54305 samples, 0 overlaps, 21 duplicate groups checked, 0 crossings	Pass

ID	Test case	Observed result	Status
TC-05	Generate a training budget	B20..B100 counts match (B100=38105, B20=7622, B40=15240, B60=22865, B80=30483), all from train	Pass
TC-06	Apply augmentation	224x224 RGB output, label 'Apple___Apple_scab' retained	Pass
TC-07	Run model inference	output (1, 38), argmax=4 p=0.026	Pass
TC-08	Generate Grad-CAM	heatmap 224x224 for class 4, finite	Pass
TC-09	Open a page without logging in	Redirected to the login page	Pass
TC-10	Login with a wrong password	“Invalid email or password” shown	Pass
TC-11	Register an existing email	“Account already exists” shown	Pass
TC-12	Upload Tomato Late blight leaf	Correct class, Grad-CAM, report, history	Pass
TC-13	Upload healthy Apple leaf	Leaf reported as healthy	Pass
TC-14	Upload a corrupt .jpg file	“Not a readable image” shown	Pass
TC-15	Upload a .txt file	“Unsupported file type” shown	Pass
TC-16	Open another user's result URL	HTTP 404 – access denied	Pass

Table 8.1 Test case summary

8.3 RESULTS AND DISCUSSION

8.3.1 Training Behaviour

Figure 8.3.1 shows the training and validation curves. Training loss fell from 0.862 to 0.054 and validation accuracy rose from 93.28% to 98.06% over 5 epochs. The validation loss remained below or close to the training loss throughout, indicating that the augmentation and weight decay prevented overfitting. The best checkpoint (epoch 5) had a validation macro F1-score of 97.50%.

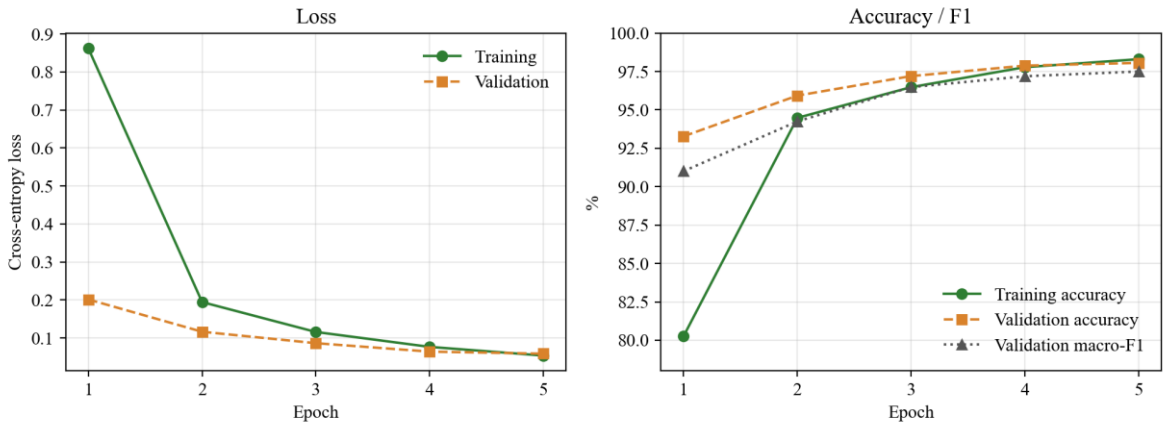


Fig 8.3.1 Training and validation curves

8.3.2 Classification Performance

Metric	Value
Test images	8,050
Accuracy	98.30%
Macro precision	97.74%
Macro recall	97.78%
Macro F1-score	97.74%
Parameters	1,556,806
Checkpoint size	6.36 MB
CPU inference time	22.0 ms / image

Table 8.2 Test-set performance of the proposed model

The model correctly classified 98.30% of the 8,050 held-out test images. Because macro F1-score (97.74%) is close to accuracy, performance is balanced across classes rather than driven by the large classes. Figure 8.3.2 shows the F1-score of every class: 34 of the 38 classes exceed 95%. The weakest classes are Tomato – Early blight (86.9%), Corn (maize) – Cercospora leaf spot Gray leaf spot (89.9%), Tomato – Target Spot (93.5%), Corn (maize) – Northern Leaf Blight (93.8%).

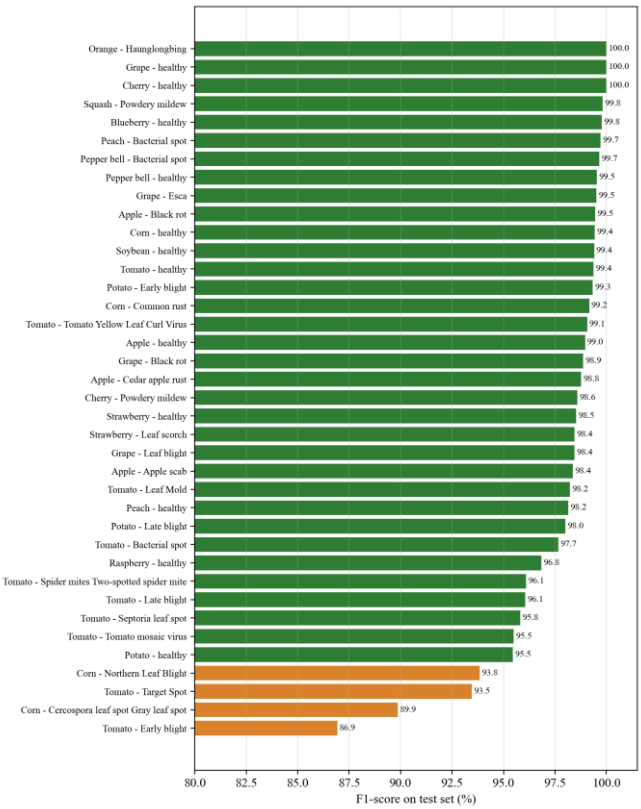


Fig 8.3.2 Per-class F1-score on the test set

8.3.3 Confusion Matrix

The confusion matrix in Figure 8.3.3 is strongly diagonal. The most frequent errors are between visually similar diseases of the same crop: Tomato – Late blight predicted as Tomato – Early blight (11 images); Corn (maize) – Northern Leaf Blight predicted as Corn (maize) – Cercospora leaf spot Gray leaf spot (9 images); Tomato – Early blight predicted as Tomato – Target Spot (8 images). These diseases produce brown necrotic lesions that are difficult to separate even for human observers, and the errors suggest where additional training images would help most.

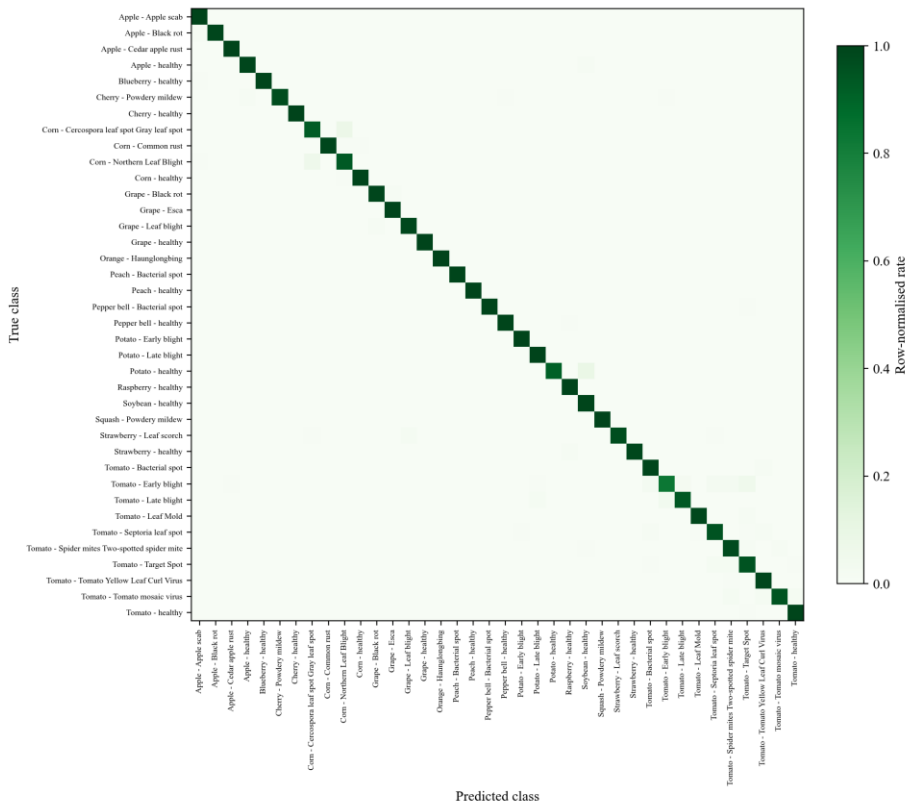


Fig 8.3.3 Confusion matrix (row-normalised)

8.3.4 Explanation Quality and Stability

Figure 8.3.4 shows Grad-CAM explanations for test images of six classes. For diseased leaves, the highest activations lie on the lesions, rust pustules and blighted tissue; for the healthy leaf the activation spreads over the leaf blade. Table 8.3 reports the measured stability of these explanations.

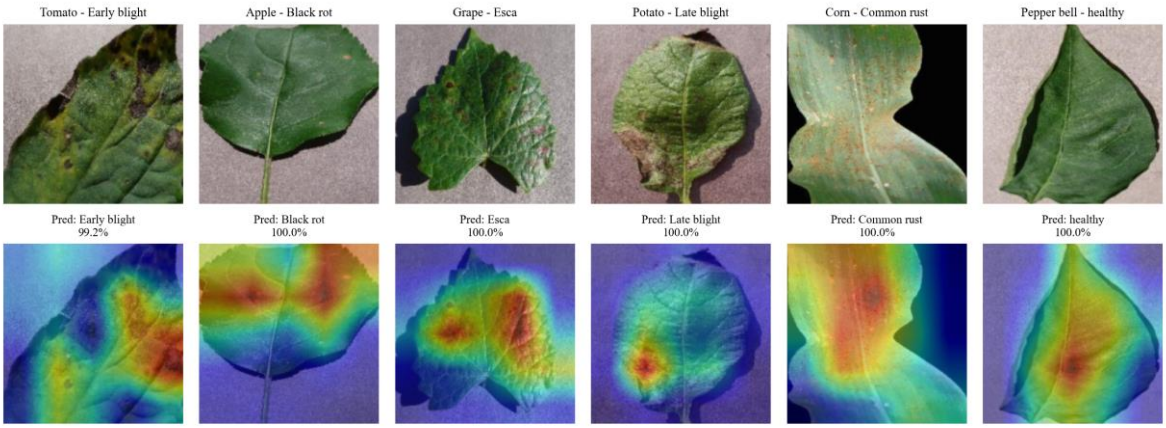


Fig 8.3.4 Grad-CAM explanations for test images

View	Prediction agreement	Mean Pearson r	Mean top-20% IoU
Horizontal flip	98.4%	0.947	0.723
Brightness +15%	99.5%	0.993	0.911
Rotation 10°	91.8%	0.809	0.597

Table 8.3 Explanation stability (380 test images)

The explanations are highly stable under brightness change (r = 0.993) and horizontal flip (r = 0.947), with prediction agreement of 99.5% and 98.4% respectively. Rotation by 10° is the hardest view (agreement 91.8%, r = 0.809) because rotation also introduces black corners and moves the leaf boundary, which the model never saw during training. This identifies rotation augmentation as a useful future improvement.

8.3.5 Comparison with Published Work

Table 8.4 places the result beside published PlantVillage studies. The published values were reported by their authors under their own data splits, so the comparison is indicative only. The proposed model uses far fewer parameters and only 40% of the training data, and is evaluated on a leaf-group-aware split that prevents leakage of near-identical images.

Study	Model	Training data	Reported accuracy
Mohanty et al. (2016)	GoogLeNet	80% of images	99.35%
Too et al. (2019)	DenseNet-121	Full training set	99.75%
Pal et al. (2025)	Mob-Res (3.51 M params)	Full training set	99.47%
Proposed system	MobileNetV3-Small (1.56 M)	40% budget (15,240 images)	98.30%

Table 8.4 Comparison with published PlantVillage results

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENT

9.1 CONCLUSION

This project developed a lightweight, explainable and web-deployed system for plant disease detection from leaf images. The PlantVillage dataset was audited and split by leaf groups to prevent leakage, and a controlled 40% training budget was used to reflect limited labelled data. A MobileNetV3-Small network with 1.56 million parameters, trained with label-preserving augmentation and class-weighted loss on a CPU, achieved 98.30% accuracy and 97.74% macro F1-score on 8,050 unseen test images, with an inference time of about 22 ms per image.

Grad-CAM explanations highlight the diseased regions and were shown to be stable under flipping and brightness changes. The Flask application makes the model usable by non-experts through secure accounts, validated uploads, visual explanations, printable reports and stored history. The system provides fast preliminary screening that can help farmers seek expert advice earlier; it does not replace confirmation by a plant pathologist.

9.2 FUTURE ENHANCEMENT

- Collect and test on field photographs with complex backgrounds to measure real-world generalisation, and add background-robust augmentation.
- Add rotation and perspective augmentation to improve stability under rotated views.
- Complete the full data-budget study (B20–B100) with repeated seeds and report mean and standard deviation.
- Add an out-of-distribution detector to reject images that are not one of the supported leaves.

- Export the model to ONNX / TensorFlow Lite and build an offline Android application.
- Provide multilingual (Tamil and English) results and link predictions to verified advisory information from agricultural departments.

APPENDIX 1

SOURCE CODE

A1.1 APP.PY (Flask web application)

```
# -*- coding: utf-8 -*-
"""Plant Disease Detection - Flask web application.

Modules served by this file:
  * User Authentication  - register / login / logout (hashed passwords, SQLite)
  * Image Upload         - validated leaf image upload
  * Prediction           - MobileNetV3-Small inference, top-5 classes
  * Explanation          - Grad-CAM heatmap of the predicted class
  * Report & History     - per-user prediction history and printable report

Run:
  python app.py          ->  http://127.0.0.1:5000
"""
import os
import sqlite3
import sys
import time
import uuid
from functools import wraps

import torch
from flask import (Flask, abort, flash, g, redirect, render_template, request,
                   session, url_for)
from PIL import Image, UnidentifiedImageError
from torchvision import transforms
from werkzeug.security import check_password_hash, generate_password_hash

ROOT = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(ROOT, "src"))
from gradcam import GradCAM, overlay  # noqa: E402
from predict import ToTensorNoNumpy, load_bundle  # noqa: E402

INSTANCE = os.path.join(ROOT, "instance")
UPLOADS = os.path.join(ROOT, "static", "uploads")
DB_PATH = os.environ.get("PLANT_DB", os.path.join(INSTANCE,
"plant_disease.db"))
ALLOWED = {"jpg", "jpeg", "png", "bmp", "webp"}
```

```
LOW_CONFIDENCE = 0.50
os.makedirs(INSTANCE, exist_ok=True)
os.makedirs(UPLOADS, exist_ok=True)


def _secret_key():
    path = os.path.join(INSTANCE, "secret_key")
    if not os.path.exists(path):
        with open(path, "wb") as fh:
            fh.write(os.urandom(32))
    with open(path, "rb") as fh:
        return fh.read()


app = Flask(__name__)
app.config.update(SECRET_KEY=_secret_key(),
                  MAX_CONTENT_LENGTH=8 * 1024 * 1024)

# ---------------------------------------------------------------- model
NET, CLASSES, CFG = load_bundle()
SIZE = int(CFG["input_size"])
PREPROCESS = transforms.Compose([
    transforms.Resize(int(SIZE * 1.15)),
    transforms.CenterCrop(SIZE),
])
TO_TENSOR = transforms.Compose([
    ToTensorNoNumpy(),
    transforms.Normalize(CFG["mean"], CFG["std"]),
])
CAM = GradCAM(NET)


def split_label(name):
    """Tomato___Late_blight' -> ('Tomato', 'Late blight')."""
    crop, _, cond = name.partition("___")
    crop = crop.replace("_", " ").replace(",", "")
    cond = cond.replace("_", " ").strip()
    return crop, cond


def is_healthy(name):
    return name.endswith("___healthy")
```

```
# ------------------------------------------------------------ database
SCHEMA = """
CREATE TABLE IF NOT EXISTS users (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    name       TEXT NOT NULL,
    email      TEXT NOT NULL UNIQUE,
    password   TEXT NOT NULL,
    created_at TEXT NOT NULL
);
CREATE TABLE IF NOT EXISTS predictions (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id     INTEGER NOT NULL REFERENCES users(id),
    image_file  TEXT NOT NULL,
    cam_file    TEXT NOT NULL,
    orig_name   TEXT NOT NULL,
    label       TEXT NOT NULL,
    confidence  REAL NOT NULL,
    top5        TEXT NOT NULL,
    latency_ms  REAL NOT NULL,
    created_at  TEXT NOT NULL
);
"""


def get_db():
    if "db" not in g:
        g.db = sqlite3.connect(DB_PATH)
        g.db.row_factory = sqlite3.Row
    return g.db


@app.teardown_appcontext
def close_db(_exc):
    db = g.pop("db", None)
    if db is not None:
        db.close()


with sqlite3.connect(DB_PATH) as _con:
    _con.executescript(SCHEMA)
```

```
def now():
    return time.strftime("%Y-%m-%d %H:%M:%S")


def login_required(view):
    @wraps(view)
    def wrapped(*args, **kwargs):
        if "user_id" not in session:
            flash("Please log in to continue.", "warning")
            return redirect(url_for("login"))
        return view(*args, **kwargs)
    return wrapped


@app.context_processor
def inject_globals():
    return {"user_name": session.get("user_name"), "cfg": CFG,
            "n_classes": len(CLASSES)}


# ---------------------------------------------------- authentication
@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        name = request.form.get("name", "").strip()
        email = request.form.get("email", "").strip().lower()
        pw = request.form.get("password", "")
        pw2 = request.form.get("confirm", "")
        if not name or "@" not in email:
            flash("Enter your name and a valid email address.", "danger")
        elif len(pw) < 6:
            flash("Password must be at least 6 characters.", "danger")
        elif pw != pw2:
            flash("Passwords do not match.", "danger")
        else:
            db = get_db()
            try:
                db.execute("INSERT INTO users (name, email, password,
created_at)"
                           " VALUES (?, ?, ?, ?)",
                           (name, email, generate_password_hash(pw), now()))
                db.commit()
            except sqlite3.IntegrityError:
```

```
            flash("An account with this email already exists.", "danger")
        else:
            flash("Registration successful. Please log in.", "success")
            return redirect(url_for("login"))
    return render_template("register.html")


@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        email = request.form.get("email", "").strip().lower()
        pw = request.form.get("password", "")
        row = get_db().execute("SELECT * FROM users WHERE email = ?",
                               (email,)).fetchone()
        if row is None or not check_password_hash(row["password"], pw):
            flash("Invalid email or password.", "danger")
        else:
            session.clear()
            session["user_id"] = row["id"]
            session["user_name"] = row["name"]
            return redirect(url_for("index"))
    return render_template("login.html")


@app.route("/logout")
def logout():
    session.clear()
    flash("You have been logged out.", "info")
    return redirect(url_for("login"))


# ---------------------------------------------- upload + prediction
@app.route("/", methods=["GET", "POST"])
@login_required
def index():
    if request.method == "POST":
        file = request.files.get("leaf")
        if file is None or file.filename == "":
            flash("Choose a leaf image to upload.", "warning")
            return redirect(url_for("index"))
        ext = file.filename.rsplit(".", 1)[-1].lower() if "." in file.filename else ""
        if ext not in ALLOWED:
```

```
            flash("Unsupported file type. Use JPG, PNG, BMP or WEBP.",
"danger")
            return redirect(url_for("index"))
        try:
            img = Image.open(file.stream)
            img.verify()
            file.stream.seek(0)
            img = Image.open(file.stream).convert("RGB")
        except (UnidentifiedImageError, OSError, SyntaxError):
            flash("The uploaded file is not a readable image.", "danger")
            return redirect(url_for("index"))
        if min(img.size) < 32:
            flash("Image is too small (minimum 32 x 32 pixels).", "danger")
            return redirect(url_for("index"))

        rid = uuid.uuid4().hex
        view = PREPROCESS(img)
        t0 = time.perf_counter()
        cam, probs = CAM(TO_TENSOR(view).unsqueeze(0))
        latency = (time.perf_counter() - t0) * 1000.0
        scores, idx = torch.topk(probs, 5)
        top5 = ";".join("%d:%.6f" % (i, s) for s, i in
                        zip(scores.tolist(), idx.tolist()))

        img_file, cam_file = rid + ".jpg", rid + "_cam.jpg"
        view.save(os.path.join(UPLOADS, img_file), quality=92)
        overlay(view, cam).save(os.path.join(UPLOADS, cam_file), quality=92)

        db = get_db()
        cur = db.execute(
            "INSERT INTO predictions (user_id, image_file, cam_file, orig_name,"
            " label, confidence, top5, latency_ms, created_at)"
            " VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)",
            (session["user_id"], img_file, cam_file, file.filename[:120],
             CLASSES[int(idx[0])], float(scores[0]), top5, latency, now()))
        db.commit()
        return redirect(url_for("result", pid=cur.lastrowid))
    return render_template("index.html")


def load_prediction(pid):
    row = get_db().execute("SELECT * FROM predictions WHERE id = ? AND
user_id = ?",
```

```
                    (pid, session["user_id"])).fetchone()
    if row is None:
        abort(404)
    top = []
    for item in row["top5"].split(";"):
        i, s = item.split(":")
        crop, cond = split_label(CLASSES[int(i)])
        top.append({"crop": crop, "cond": cond, "score": float(s)})
    crop, cond = split_label(row["label"])
    return {"row": row, "top": top, "crop": crop, "cond": cond,
            "healthy": is_healthy(row["label"]),
            "low": row["confidence"] < LOW_CONFIDENCE}


@app.route("/result/<int:pid>")
@login_required
def result(pid):
    return render_template("result.html", **load_prediction(pid))


@app.route("/report/<int:pid>")
@login_required
def report(pid):
    return render_template("report.html", **load_prediction(pid))


@app.route("/history")
@login_required
def history():
    rows = get_db().execute(
        "SELECT * FROM predictions WHERE user_id = ? ORDER BY id
DESC",
        (session["user_id"],)).fetchall()
    items = []
    for r in rows:
        crop, cond = split_label(r["label"])
        items.append({"row": r, "crop": crop, "cond": cond,
                      "healthy": is_healthy(r["label"])})
    return render_template("history.html", items=items)


@app.errorhandler(413)
def too_large(_e):
```

```
    flash("File is larger than 8 MB.", "danger")
    return redirect(url_for("index"))


if __name__ == "__main__":
    app.run(host="127.0.0.1", port=int(os.environ.get("PORT", "5000")),
debug=False)
```

A1.2 PREDICT.PY (model bundle loader)

```
# -*- coding: utf-8 -*-
"""Run a single-image prediction with the exported model bundle.

Usage:
  python src/predict.py path\\to\\leaf.jpg
  python src/predict.py path\\to\\leaf.jpg --topk 5
"""
import argparse
import json
import os

import torch
import torch.nn as nn
from PIL import Image
from torchvision import models, transforms

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
BUNDLE = os.path.join(ROOT, "model_bundle")


class ToTensorNoNumpy(object):
    """PIL -> float CHW tensor without the NumPy bridge (portable to NumPy
1.x builds)."""

    def __call__(self, img):
        if img.mode != "RGB":
            img = img.convert("RGB")
        buf = torch.frombuffer(bytearray(img.tobytes()), dtype=torch.uint8)
        return (buf.view(img.height, img.width, 3)
                .permute(2, 0, 1)
                .float()
                .div_(255.0))
```

```
def load_bundle(path=BUNDLE):
    with open(os.path.join(path, "config.json"), encoding="utf-8") as fh:
        cfg = json.load(fh)
    with open(os.path.join(path, "classes.json"), encoding="utf-8") as fh:
        classes = json.load(fh)

    net = models.mobilenet_v3_small(weights=None)
    net.classifier[3] = nn.Linear(net.classifier[3].in_features, len(classes))
    net.num_classes = len(classes)
    state = torch.load(os.path.join(path, "model.pt"), map_location="cpu",
                       weights_only=True)
    net.load_state_dict(state["state_dict"])
    net.eval()
    return net, classes, cfg


def predict(image_path, topk=5, bundle=BUNDLE):
    net, classes, cfg = load_bundle(bundle)
    size = int(cfg["input_size"])
    tf = transforms.Compose([
        transforms.Resize(int(size * 1.15)),
        transforms.CenterCrop(size),
        ToTensorNoNumpy(),
        transforms.Normalize(cfg["mean"], cfg["std"]),
    ])
    with Image.open(image_path) as im:
        x = tf(im.convert("RGB")).unsqueeze(0)
    with torch.no_grad():
        prob = torch.softmax(net(x), dim=1)[0]
    scores, idx = prob.topk(min(topk, len(classes)))
    return [(classes[i], float(s)) for s, i in zip(scores, idx)]


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("image")
    ap.add_argument("--topk", type=int, default=5)
    ap.add_argument("--bundle", default=BUNDLE)
    a = ap.parse_args()

    results = predict(a.image, a.topk, a.bundle)
    print("image :", a.image)
```

```
    for rank, (name, score) in enumerate(results, 1):
        print("%d. %-55s %.2f%%" % (rank, name, score * 100.0))


if __name__ == "__main__":
    main()
```

A1.3 GRADCAM.PY (explanation module)

```
# -*- coding: utf-8 -*-
"""Grad-CAM for the MobileNetV3-Small plant-disease classifier.

Grad-CAM (Selvaraju et al., 2017) weights each feature map A^k of the last
convolutional block by the spatial mean of d y^c / d A^k, sums the weighted
maps and keeps the positive part. The resulting coarse map is resized to the
input image and blended over it as a heatmap.
"""
import torch
from matplotlib import colormaps
from PIL import Image


class GradCAM(object):
    def __init__(self, model, target_layer=None):
        self.model = model
        self.layer = target_layer if target_layer is not None else model.features[-1]
        self._acts = None
        self._grads = None
        self.layer.register_forward_hook(self._save_acts)
        self.layer.register_full_backward_hook(self._save_grads)

    def _save_acts(self, _module, _inp, out):
        self._acts = out.detach()

    def _save_grads(self, _module, _gin, gout):
        self._grads = gout[0].detach()

    def __call__(self, x, class_idx=None):
        """Return (cam[H', W'] in 0..1, probabilities[C]) for one image tensor."""
        self.model.zero_grad(set_to_none=True)
        with torch.enable_grad():
            logits = self.model(x)
            probs = torch.softmax(logits, dim=1)[0].detach()
```

```
            if class_idx is None:
                class_idx = int(probs.argmax())
            logits[0, class_idx].backward()
        weights = self._grads.mean(dim=(2, 3), keepdim=True)          # alpha_k^c
        cam = torch.relu((weights * self._acts).sum(dim=1))[0]        #
ReLU(sum_k)
        cam = cam - cam.min()
        if float(cam.max()) > 0:
            cam = cam / cam.max()
        return cam, probs


def overlay(img, cam, alpha=0.45):
    """Blend a 0..1 CAM over a PIL RGB image using the 'jet' colour map."""
    w, h = img.size
    cam_img = Image.fromarray(
        (cam.mul(255).byte().cpu().numpy()), mode="L").resize((w, h),
Image.BILINEAR)
    jet = colormaps["jet"]
    lut = [tuple(int(255 * v) for v in jet(i / 255.0)[:3]) for i in range(256)]
    heat = Image.new("RGB", (w, h))
    heat.putdata([lut[p] for p in cam_img.getdata()])
    return Image.blend(img.convert("RGB"), heat, alpha)
```

A1.4 TRAIN.PY (training pipeline)

```
# -*- coding: utf-8 -*-
"""Train the demo classifier used by the inference app.

Follows the experiment protocol declared in the report:
  * fixed leaf-group-aware split from split_manifest.csv (seed 42)
  * per-class budgets sampled from the TRAINING partition only
  * MobileNetV3-Small backbone with a 38-class head
  * class-weighted cross entropy + label-preserving augmentation
  * best checkpoint selected on validation macro-F1, final report on test

Usage:
  python src/train.py --bench               # 12-iteration speed probe
  python src/train.py --budget B40 --epochs 12
"""
import argparse
import csv
import json
```

```
import os
import random
import time
from collections import defaultdict

import torch
import torch.nn as nn
from PIL import Image, ImageEnhance
from torch.utils.data import DataLoader, Dataset
from torchvision import models, transforms

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
SPLITS = os.path.join(ROOT, "dataset", "PlantVillage", "splits")
RAW = os.path.join(ROOT, "dataset", "PlantVillage", "raw", "color")
ART = os.path.join(ROOT, "artifacts")
CKPT_DIR = os.path.join(ART, "checkpoints")
METRIC_DIR = os.path.join(ART, "metrics")
BUNDLE = os.path.join(ROOT, "model_bundle")
# Keep the torch hub cache inside the project: the default ~/.cache path is not
# writable in this environment, which made the pretrained download fail early.
os.environ.setdefault("TORCH_HOME", os.path.join(ART, "torch_cache"))
os.makedirs(CKPT_DIR, exist_ok=True)
os.makedirs(METRIC_DIR, exist_ok=True)


# ---------------------------------------------------------------- utils
def seed_everything(s=42):
    random.seed(s)
    torch.manual_seed(s)


def load_rows():
    with open(os.path.join(SPLITS, "split_manifest.csv"), encoding="utf-8") as
fh:
        return list(csv.DictReader(fh))


def build_class_index(rows):
    classes = sorted({r["class_name"] for r in rows})
    return classes, {c: i for i, c in enumerate(classes)}


def partition_keys(rows, partition, budget=None):
```

```
    """Keys for one partition; optionally restricted to a training budget."""
    budget_keys = None
    if budget:
        path = os.path.join(SPLITS, "budgets", "%s_manifest.csv" % budget)
        with open(path, encoding="utf-8") as fh:
            budget_keys = {(r["class_name"], r["file_name"])
                           for r in csv.DictReader(fh)}
    out = []
    for r in rows:
        if r["partition"] != partition:
            continue
        key = (r["class_name"], r["file_name"])
        if budget_keys is not None and key not in budget_keys:
            continue
        out.append(key)
    return out


def class_counts(keys):
    cnt = defaultdict(int)
    for c, _ in keys:
        cnt[c] += 1
    return cnt


def build_model(n_cls, pretrained=True, log=print):
    """MobileNetV3-Small with a fresh 38-way head.

    ``num_classes`` cannot be overridden while requesting pretrained weights
    (torchvision raises ValueError: expected 1000). Build the ImageNet model
    first and then swap the final classifier layer. When the weights cannot be
    downloaded (offline machine), fall back to random initialisation instead of
    crashing the run.
    """
    def _assemble(weights):
        net = models.mobilenet_v3_small(weights=weights)
        in_features = net.classifier[3].in_features
        net.classifier[3] = nn.Linear(in_features, n_cls)
        net.num_classes = n_cls
        return net

    if pretrained:
        try:
```

```
            return
_assemble(models.MobileNet_V3_Small_Weights.IMAGENET1K_V1), True
        except Exception as exc:  # URLError / HTTPError / socket errors
            log("WARNING: pretrained weights unavailable (%s: %s); "
                "continuing with random initialisation."
                % (type(exc).__name__, exc))
    return _assemble(None), False


# ---------------------------------------------------------------- dataset
class LeafDataset(Dataset):
    def __init__(self, keys, cls_idx, tf):
        self.keys = keys
        self.cls_idx = cls_idx
        self.tf = tf

    def __len__(self):
        return len(self.keys)

    def __getitem__(self, i):
        cls, fn = self.keys[i]
        path = os.path.join(RAW, cls, fn)
        with Image.open(path) as im:
            img = im.convert("RGB")
        return self.tf(img), self.cls_idx[cls]


class ToTensorNoNumpy(object):
    """PIL -> float CHW tensor without going through the NumPy bridge.

    Kept NumPy-free so the pipeline also works on torch builds that were
    compiled against NumPy 1.x (where ``transforms.ToTensor`` can raise
    "Numpy is not available"). The result is identical to ``ToTensor``.
    """

    def __call__(self, img):
        if img.mode != "RGB":
            img = img.convert("RGB")
        buf = torch.frombuffer(bytearray(img.tobytes()), dtype=torch.uint8)
        return (buf.view(img.height, img.width, 3)
                .permute(2, 0, 1)
                .float()
                .div_(255.0))
```

```
class RandomColorJitter(object):
    """Brightness/contrast/saturation/hue jitter implemented with PIL only.

    Avoids torchvision's ``np.uint8`` hue arithmetic (which raises
    ``OverflowError`` under NumPy 2.x) and keeps augmentation independent of
    the NumPy ABI. The result is label-preserving, like ``ColorJitter``.
    """

    def __init__(self, brightness=0.25, contrast=0.25, saturation=0.25,
                 hue=0.10):
        self.brightness = brightness
        self.contrast = contrast
        self.saturation = saturation
        self.hue = hue

    @staticmethod
    def _factor(value):
        return random.uniform(max(0.0, 1.0 - value), 1.0 + value)

    @staticmethod
    def _adjust_hue(img, factor):
        if factor == 0.0:
            return img
        hsv = img.convert("HSV")
        h, s, v = hsv.split()
        delta = int(factor * 255)
        lut = [(i + delta) % 256 for i in range(256)]
        return Image.merge("HSV", (h.point(lut), s, v)).convert("RGB")

    def __call__(self, img):
        if self.brightness:
            img =
ImageEnhance.Brightness(img).enhance(self._factor(self.brightness))
        if self.contrast:
            img = ImageEnhance.Contrast(img).enhance(self._factor(self.contrast))
        if self.saturation:
            img = ImageEnhance.Color(img).enhance(self._factor(self.saturation))
        if self.hue:
            img = self._adjust_hue(
                img, random.uniform(-self.hue, self.hue))
        return img
```

```
def make_transforms(size):
    mean = [0.485, 0.456, 0.406]
    std = [0.229, 0.224, 0.225]
    train_tf = transforms.Compose([
        transforms.RandomResizedCrop(size, scale=(0.6, 1.0)),
        transforms.RandomHorizontalFlip(),
        RandomColorJitter(0.25, 0.25, 0.25, 0.10),
        ToTensorNoNumpy(),
        transforms.Normalize(mean, std),
    ])
    eval_tf = transforms.Compose([
        transforms.Resize(int(size * 1.15)),
        transforms.CenterCrop(size),
        ToTensorNoNumpy(),
        transforms.Normalize(mean, std),
    ])
    return train_tf, eval_tf, mean, std


# ---------------------------------------------------------------------- metrics
def macro_f1_and_acc(preds, golds, n_cls):
    tp = [0] * n_cls
    fp = [0] * n_cls
    fn = [0] * n_cls
    correct = 0
    for p, g in zip(preds, golds):
        if p == g:
            tp[p] += 1
            correct += 1
        else:
            fp[p] += 1
            fn[g] += 1
    f1s = []
    for c in range(n_cls):
        prec = tp[c] / (tp[c] + fp[c]) if (tp[c] + fp[c]) else 0.0
        rec = tp[c] / (tp[c] + fn[c]) if (tp[c] + fn[c]) else 0.0
        f1s.append(2 * prec * rec / (prec + rec) if (prec + rec) else 0.0)
    acc = correct / len(golds) if golds else 0.0
    macro = sum(f1s) / n_cls if n_cls else 0.0
    return acc, macro, f1s
```

```
@torch.no_grad()
def evaluate(model, loader, device):
    model.eval()
    preds, golds, losses = [], [], []
    crit = nn.CrossEntropyLoss(reduction="sum")
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        x = x.to(memory_format=torch.channels_last)
        logits = model(x)
        losses.append(float(crit(logits, y)))
        preds.extend(logits.argmax(1).tolist())
        golds.extend(y.tolist())
    n = len(golds)
    acc, macro, f1s = macro_f1_and_acc(preds, golds, model.num_classes)
    loss = sum(losses) / n if n else 0.0
    return {"loss": loss, "acc": acc, "macro_f1": macro, "f1": f1s,
            "preds": preds, "golds": golds}


# ---------------------------------------------------------------- training
def parse_args():
    p = argparse.ArgumentParser()
    p.add_argument("--budget", default="B40",
                   choices=["B20", "B40", "B60", "B80", "B100", "all"])
    p.add_argument("--epochs", type=int, default=12)
    p.add_argument("--img", type=int, default=160)
    p.add_argument("--batch", type=int, default=32)
    p.add_argument("--lr", type=float, default=3e-4)
    p.add_argument("--workers", type=int, default=0,
                   help="DataLoader worker processes; 0 is required under "
                        "sandboxed Windows where named pipes are blocked")
    p.add_argument("--patience", type=int, default=4)
    p.add_argument("--seed", type=int, default=42)
    p.add_argument("--bench", action="store_true")
    p.add_argument("--tag", default="demo")
    return p.parse_args()


def main():
    a = parse_args()
    seed_everything(a.seed)
    device = torch.device("cpu")
```

```
    torch.set_num_threads(min(12, os.cpu_count() or 4))

    t0 = time.time()
    rows = load_rows()
    classes, cls_idx = build_class_index(rows)
    n_cls = len(classes)
    budget = None if a.budget == "all" else a.budget

    train_keys = partition_keys(rows, "train", budget)
    val_keys = partition_keys(rows, "validation")
    test_keys = partition_keys(rows, "test")
    print("classes=%d  train=%d  val=%d  test=%d  budget=%s"
          % (n_cls, len(train_keys), len(val_keys), len(test_keys),
             budget or "all"))

    train_tf, eval_tf, mean, std = make_transforms(a.img)
    train_ds = LeafDataset(train_keys, cls_idx, train_tf)
    val_ds = LeafDataset(val_keys, cls_idx, eval_tf)
    test_ds = LeafDataset(test_keys, cls_idx, eval_tf)

    if a.bench:
        loader = DataLoader(train_ds, batch_size=a.batch, shuffle=False,
                            num_workers=0)
        model, pretrained = build_model(n_cls, pretrained=True)
        model = model.to(memory_format=torch.channels_last)
        print("backbone=%s pretrained=%s" % ("MobileNetV3-Small",
pretrained))
        model.train()
        opt = torch.optim.AdamW(model.parameters(), lr=a.lr)
        crit = nn.CrossEntropyLoss()
        it = iter(loader)
        n_it = 0
        t1 = time.time()
        while n_it < 12:
            try:
                x, y = next(it)
            except StopIteration:
                it = iter(loader)
                x, y = next(it)
            opt.zero_grad()
            x = x.to(memory_format=torch.channels_last)
            loss = crit(model(x), y)
            loss.backward()
```

```
            opt.step()
            n_it += 1
        dt = (time.time() - t1) / n_it
        per_epoch = dt * len(train_ds) / a.batch
        print("BENCH: %.2f s/iter  ->  %.1f min/epoch  ->  %d epochs ~ %.0f
min"
              % (dt, per_epoch / 60, a.epochs, per_epoch * a.epochs / 60))
        return

    # class-weighted loss (inverse sqrt frequency, normalised to mean 1)
    cnt = class_counts(train_keys)
    w = [1.0 / (cnt[c] ** 0.5) for c in classes]
    mean_w = sum(w) / len(w)
    weight = torch.tensor([x / mean_w for x in w], dtype=torch.float32)
    crit = nn.CrossEntropyLoss(weight=weight)

    model, pretrained = build_model(n_cls, pretrained=True)
    model = model.to(memory_format=torch.channels_last)
    print("backbone=MobileNetV3-Small  pretrained=%s" % pretrained)
    opt = torch.optim.AdamW(model.parameters(), lr=a.lr, weight_decay=1e-4)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(
        opt, T_max=max(1, a.epochs))

    train_loader = DataLoader(train_ds, batch_size=a.batch, shuffle=True,
                              num_workers=a.workers, drop_last=False)
    val_loader = DataLoader(val_ds, batch_size=a.batch * 2, shuffle=False,
                            num_workers=a.workers)

    log_path = os.path.join(ART, "training_log_%s.csv" % a.tag)
    with open(log_path, "w", newline="", encoding="utf-8") as fh:
        csv.writer(fh).writerow(
            ["epoch", "train_loss", "train_acc", "val_loss", "val_acc",
             "val_macro_f1", "lr", "seconds"])

    best = {"macro_f1": -1.0}
    bad = 0
    for ep in range(1, a.epochs + 1):
        t1 = time.time()
        model.train()
        run_loss, run_ok, run_n = 0.0, 0, 0
        for x, y in train_loader:
            opt.zero_grad()
            x = x.to(memory_format=torch.channels_last)
```

```
            logits = model(x)
            loss = crit(logits, y)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
            run_loss += float(loss) * len(y)
            run_ok += int((logits.argmax(1) == y).sum())
            run_n += len(y)
        sched.step()

        val = evaluate(model, val_loader, device)
        secs = time.time() - t1
        tr_loss, tr_acc = run_loss / run_n, run_ok / run_n
        lr_now = opt.param_groups[0]["lr"]
        print("ep %2d/%d  train_loss %.3f acc %.3f | val_loss %.3f "
              "acc %.3f macroF1 %.3f | %.0fs"
              % (ep, a.epochs, tr_loss, tr_acc, val["loss"], val["acc"],
                 val["macro_f1"], secs), flush=True)
        with open(log_path, "a", newline="", encoding="utf-8") as fh:
            csv.writer(fh).writerow(
                [ep, round(tr_loss, 4), round(tr_acc, 4),
                 round(val["loss"], 4), round(val["acc"], 4),
                 round(val["macro_f1"], 4), round(lr_now, 6), int(secs)])

        if val["macro_f1"] > best["macro_f1"] + 1e-4:
            best = {"macro_f1": val["macro_f1"], "acc": val["acc"],
                    "loss": val["loss"], "epoch": ep}
            bad = 0
            torch.save({"state_dict": model.state_dict(), "classes": classes,
                        "img": a.img, "budget": budget or "B100",
                        "seed": a.seed, "epoch": ep, "pretrained": pretrained,
                        "val": {"acc": val["acc"],
                                "macro_f1": val["macro_f1"]}},
                       os.path.join(CKPT_DIR, "model_best.pt"))
        else:
            bad += 1
            if bad >= a.patience:
                print("early stop at epoch %d (best epoch %d)"
                      % (ep, best["epoch"]))
                break

# ---------------------------------------------------------- final test
bundle_path = os.path.join(CKPT_DIR, "model_best.pt")
```

```
    state = torch.load(bundle_path, map_location="cpu", weights_only=False)
    model.load_state_dict(state["state_dict"])
    test_loader = DataLoader(test_ds, batch_size=a.batch * 2, shuffle=False,
                             num_workers=a.workers)
    test = evaluate(model, test_loader, device)

    n_params = sum(p.numel() for p in model.parameters())
    ckpt_mb = os.path.getsize(bundle_path) / 1e6
    metrics = {
        "budget": budget or "B100",
        "seed": a.seed,
        "input_size": a.img,
        "classes": n_cls,
        "train_images": len(train_keys),
        "val_images": len(val_keys),
        "test_images": len(test_keys),
        "best_epoch": best["epoch"],
        "val_acc": round(best["acc"], 4),
        "val_macro_f1": round(best["macro_f1"], 4),
        "test_acc": round(test["acc"], 4),
        "test_macro_f1": round(test["macro_f1"], 4),
        "per_class_f1": {c: round(v, 4) for c, v in zip(classes, test["f1"])},
        "parameters": n_params,
        "pretrained": pretrained,
        "checkpoint_mb": round(ckpt_mb, 2),
        "wall_minutes": round((time.time() - t0) / 60, 1),
        "device": "cpu",
    }
    with open(os.path.join(METRIC_DIR, "metrics_%s.json" % a.tag), "w",
              encoding="utf-8") as fh:
        json.dump(metrics, fh, indent=2)

    # ------------------------------------------------------- export bundle
    os.makedirs(BUNDLE, exist_ok=True)
    torch.save({"state_dict": model.state_dict()}, os.path.join(BUNDLE,
"model.pt"))
    with open(os.path.join(BUNDLE, "classes.json"), "w", encoding="utf-8") as
fh:
        json.dump(classes, fh, indent=2)
    with open(os.path.join(BUNDLE, "config.json"), "w", encoding="utf-8") as
fh:
        json.dump({"input_size": a.img, "mean": mean, "std": std,
                   "backbone": "MobileNetV3-Small", "classes": n_cls,
```

```
                "budget": budget or "B100", "seed": a.seed,
                "pretrained": pretrained,
                "test_acc": metrics["test_acc"],
                "test_macro_f1": metrics["test_macro_f1"],
                "trained_at": time.strftime("%Y-%m-%d %H:%M:%S")}, fh,
indent=2)

    print("\nTEST acc %.4f  macro-F1 %.4f  (%.1f min, %.1f MB)"
          % (test["acc"], test["macro_f1"], metrics["wall_minutes"], ckpt_mb))
    print("bundle ->", BUNDLE)


if __name__ == "__main__":
    main()
```

A1.5 RESULT.HTML (result page template)

```
{% extends "base.html" %}
{% block title %}Result &middot; Plant Disease Detection{% endblock %}
{% block content %}
<section class="card verdict {{ 'ok' if healthy else 'bad' }}">
  <div>
    <p class="eyebrow">Prediction</p>
    <h1>{{ crop }} &ndash; {{ cond }}</h1>
    <p>{% if healthy %}The leaf appears <strong>healthy</strong>.{% else
%}Signs of
       <strong>{{ cond }}</strong> detected on {{ crop }}.{% endif %}
       Confidence <strong>{{ '%.2f' % (100 * row.confidence) }}
%</strong>.</p>
    {% if low %}
    <p class="warn">Low confidence: the image may be unclear or outside the
{{ n_classes }} supported
       classes. Retake the photo or consult an expert.</p>
    {% endif %}
  </div>
  <div class="actions">
    <a class="btn btn-primary" href="{{ url_for('report', pid=row.id) }}">View
report</a>
    <a class="btn btn-ghost" href="{{ url_for('index') }}">New image</a>
  </div>
</section>

<section class="grid-3">
```

```
  <figure class="card">
    <img src="{{ url_for('static', filename='uploads/' + row.image_file) }}"
alt="Uploaded leaf">
    <figcaption>Input ({{ row.orig_name }})</figcaption>
  </figure>
  <figure class="card">
    <img src="{{ url_for('static', filename='uploads/' + row.cam_file) }}"
alt="Grad-CAM heatmap">
    <figcaption>Grad-CAM: red = regions that most influenced the
prediction</figcaption>
  </figure>
  <div class="card">
    <h2>Top-5 classes</h2>
    {% for t in top %}
    <div class="bar-row">
      <div class="bar-label"><span>{{ loop.index }}. {{ t.crop }} &ndash; {{
t.cond }}</span>
        <span>{{ '%.2f' % (100 * t.score) }}%</span></div>
      <div class="bar"><div style="width: {{ '%.1f' % (100 * t.score)
}}%"></div></div>
    </div>
    {% endfor %}
    <p class="muted small">Inference + Grad-CAM time: {{ '%.0f' %
row.latency_ms }} ms (CPU)</p>
  </div>
</section>
{% endblock %}
```

APPENDIX 2

SCREENSHOTS

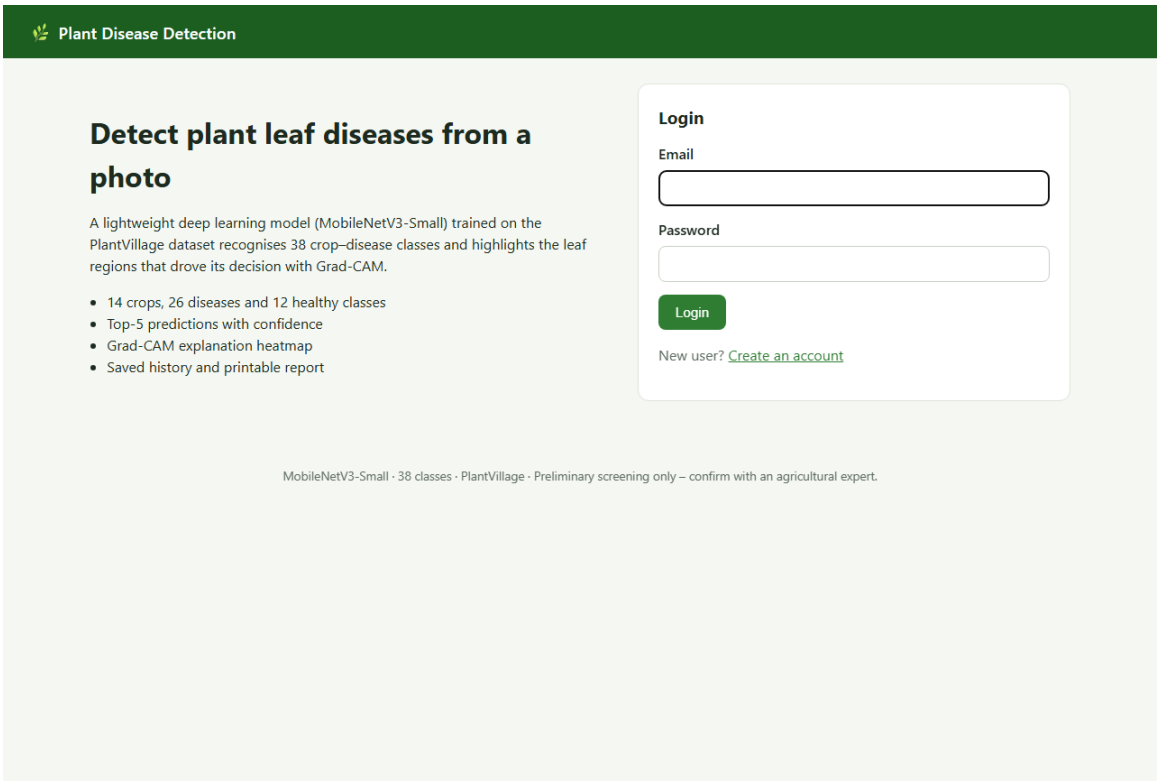


Fig A2.1 Login page

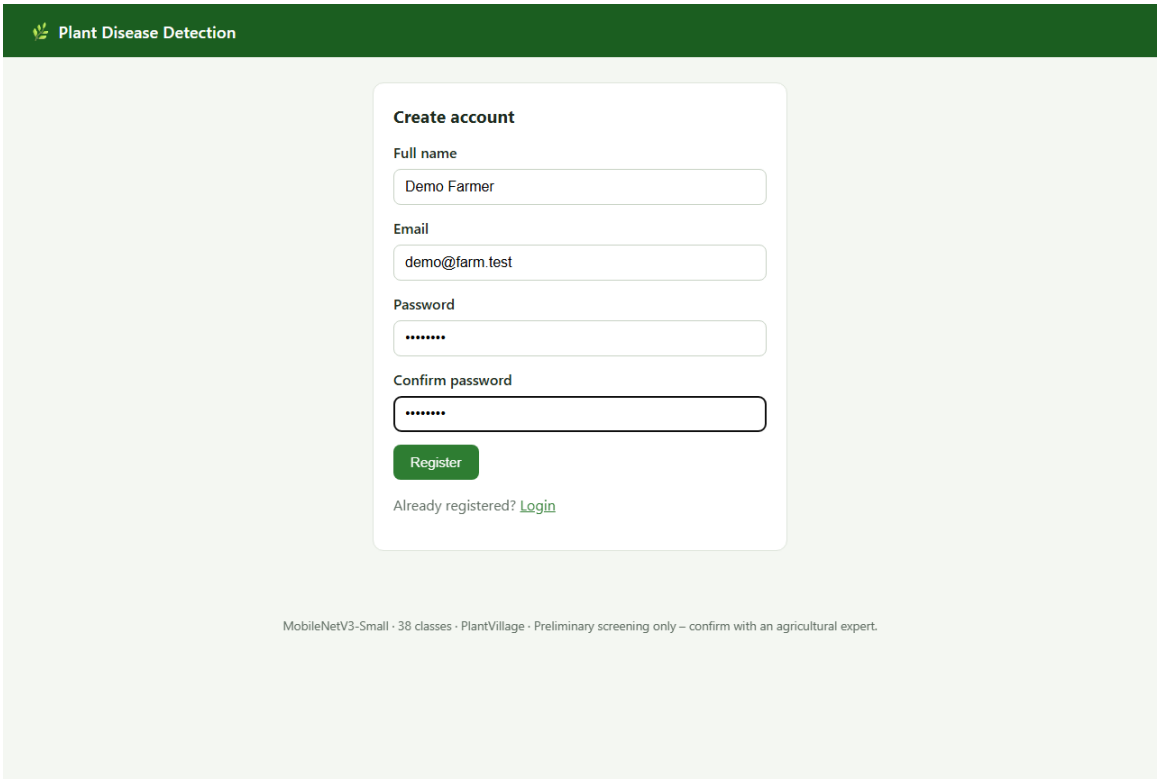


Fig A2.2 Registration page

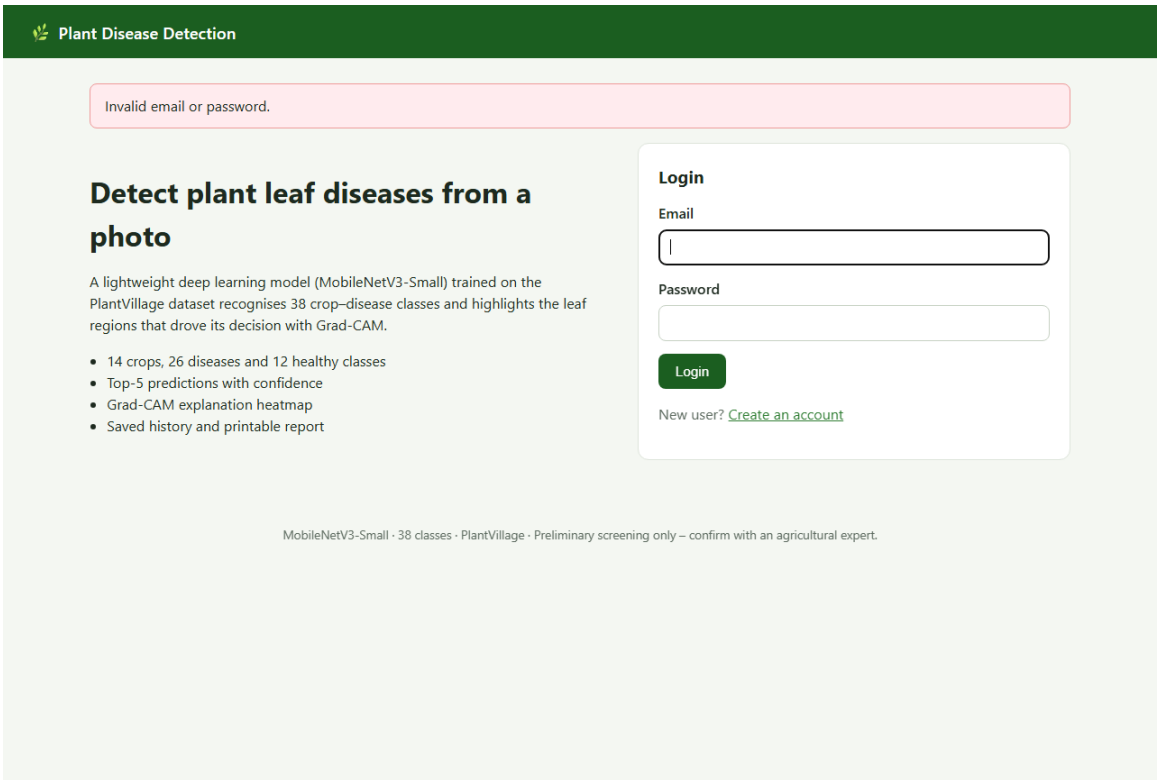


Fig A2.3 Login with invalid credentials

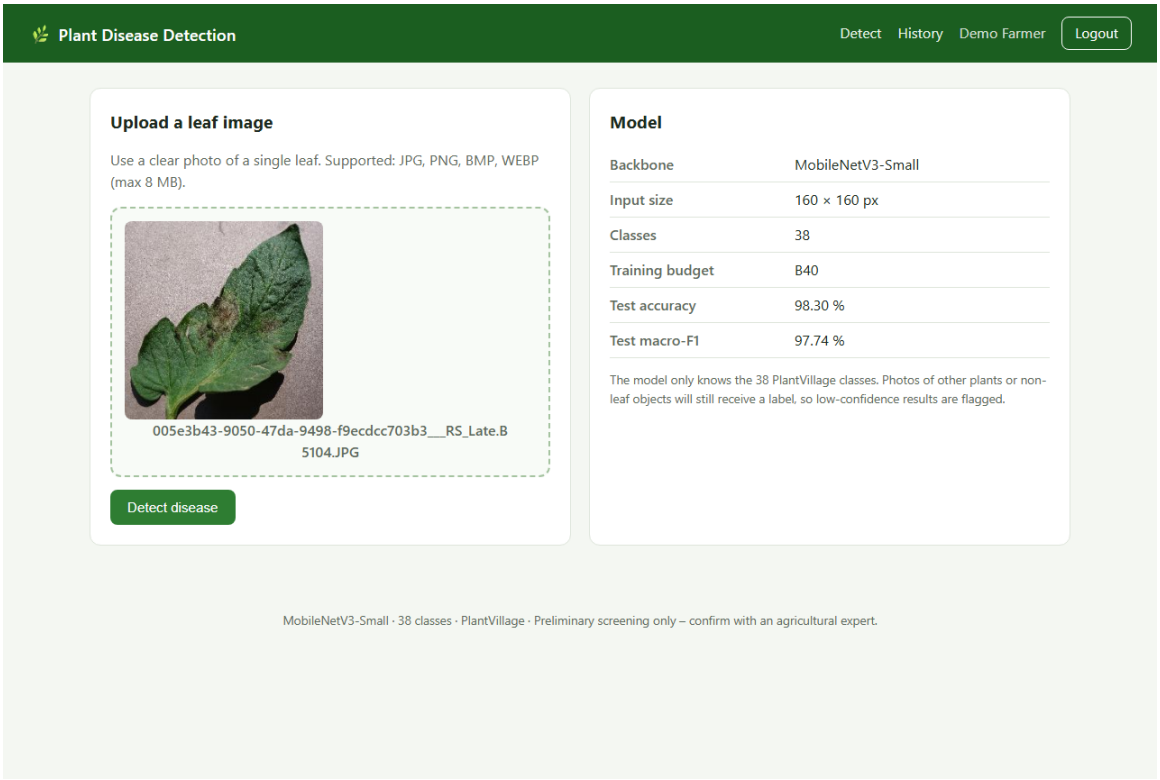


Fig A2.4 Leaf image upload page

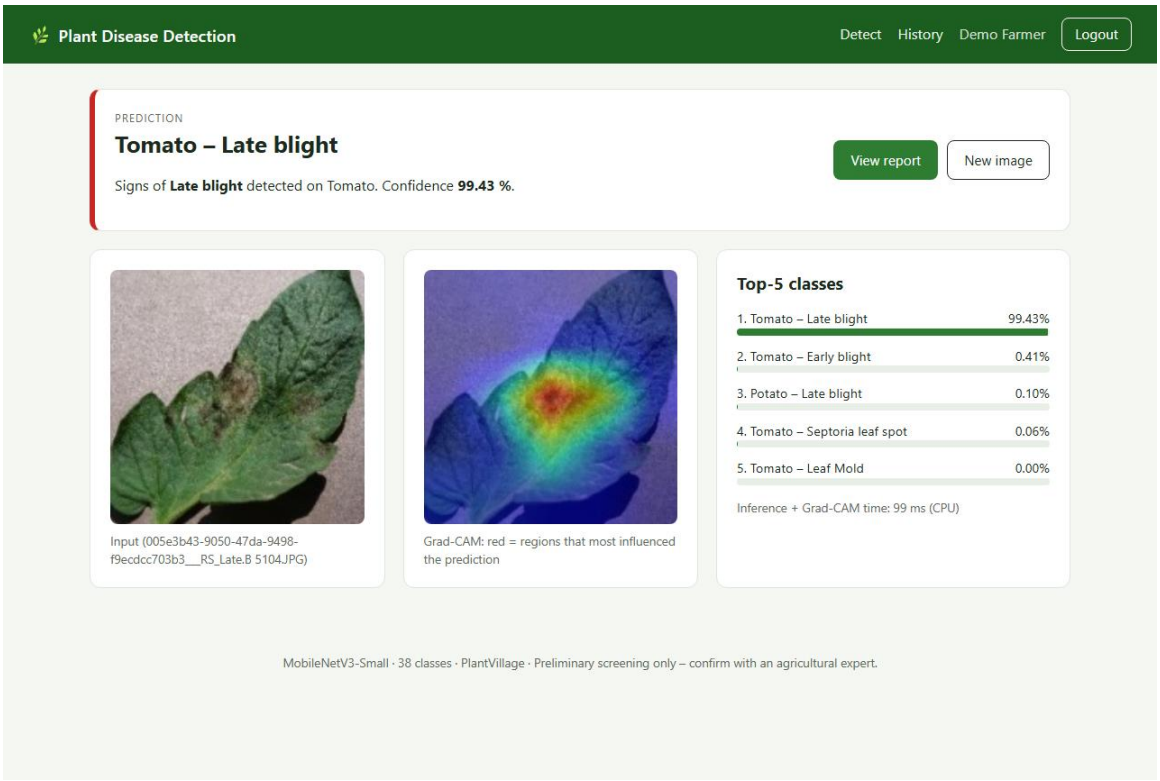


Fig A2.5 Prediction result – Tomato Late blight with Grad-CAM

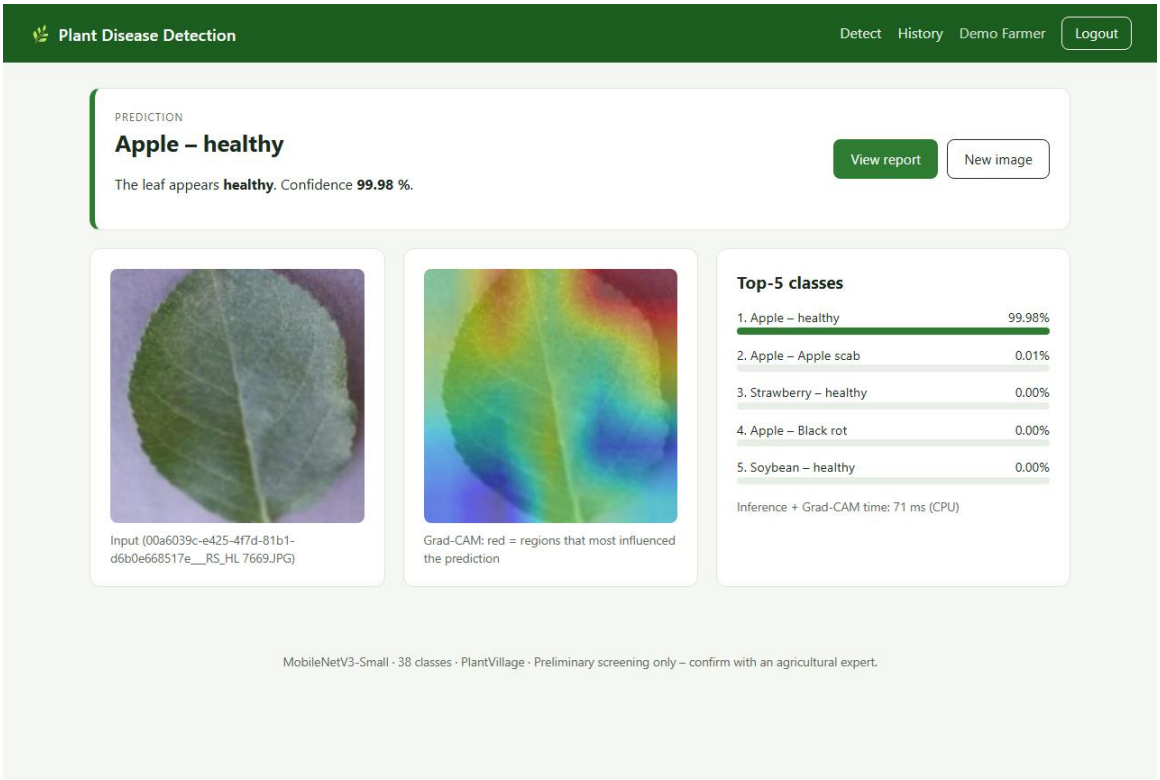


Fig A2.6 Prediction result – healthy Apple leaf

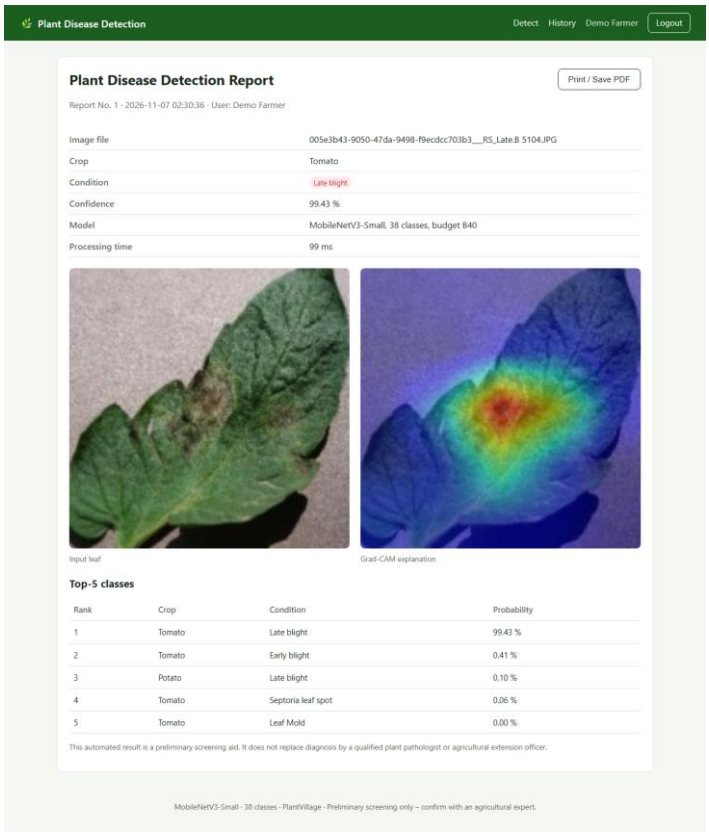


Fig A2.7 Printable prediction report

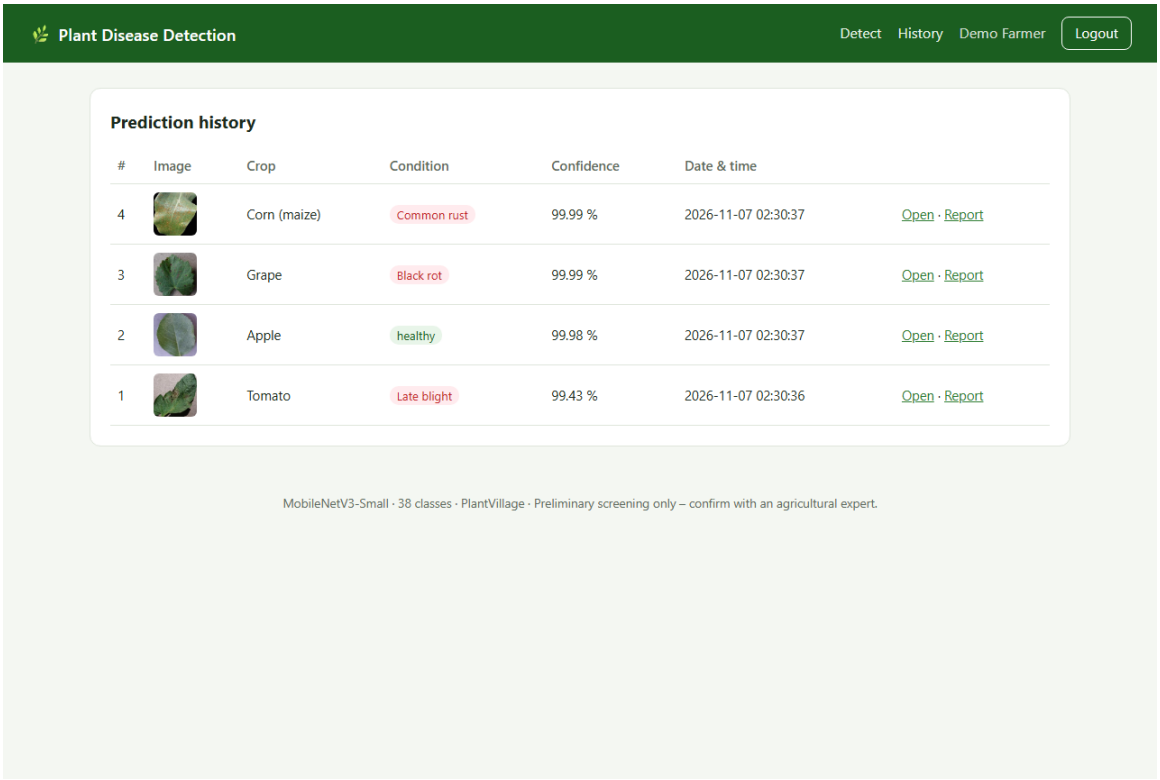


Fig A2.8 Prediction history

REFERENCES

[1] Hughes, D. P. and Salathé, M. (2015), 'An open access repository of images on plant health to enable the development of mobile disease diagnostics', arXiv:1511.08060.

[2] Mohanty, S. P., Hughes, D. P. and Salathé, M. (2016), 'Using deep learning for image-based plant disease detection', Frontiers in Plant Science, Vol. 7, Article 1419.

[3] Sladojevic, S., Arsenovic, M., Anderla, A., Culibrk, D. and Stefanovic, D. (2016), 'Deep neural networks based recognition of plant diseases by leaf image classification', Computational Intelligence and Neuroscience, Vol. 2016, Article 3289801.

[4] Brahimi, M., Boukhalfa, K. and Moussaoui, A. (2017), 'Deep learning for tomato diseases: classification and symptoms visualization', Applied Artificial Intelligence, Vol. 31, No. 4, pp. 299–315.

[5] Ferentinos, K. P. (2018), 'Deep learning models for plant disease detection and diagnosis', Computers and Electronics in Agriculture, Vol. 145, pp. 311–318.

[6] Barbedo, J. G. A. (2018), 'Impact of dataset size and variety on the effectiveness of deep learning and transfer learning for plant disease classification', Computers and Electronics in Agriculture, Vol. 153, pp. 46–53.

[7] Too, E. C., Yujian, L., Njuki, S. and Yingchun, L. (2019), 'A comparative study of fine-tuning deep learning models for plant disease identification', Computers and Electronics in Agriculture, Vol. 161, pp. 272–279.

[8] Howard, A., Sandler, M., Chu, G., Chen, L.-C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V., Le, Q. V. and Adam, H. (2019), 'Searching for MobileNetV3', Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 1314–1324.

[9] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D. (2017), 'Grad-CAM: visual explanations from deep networks via gradient-based localization', Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 618–626.

[10] Pal, C., Karmakar, S., Mukherjee, I. and Chakrabarti, P. P. (2025), 'A lightweight and explainable CNN model for empowering plant disease diagnosis', Scientific Reports, Vol. 15, Article 30720, doi:10.1038/s41598-025-94083-1.

[11] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018), 'MobileNetV2: inverted residuals and linear bottlenecks', Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4510–4520.

[12] He, K., Zhang, X., Ren, S. and Sun, J. (2016), 'Deep residual learning for image recognition', Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778.

[13] Paszke, A. et al. (2019), 'PyTorch: an imperative style, high-performance deep learning library', Advances in Neural Information Processing Systems 32, pp. 8024–8035.

[14] Pallets Projects (2025), 'Flask documentation', https://flask.palletsprojects.com/.